MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION (MUMBAI)

A

PROJECT REPORT

ON

"OBE TRACKING SYSTEM''

SUBMITTED BY

FARHEEN SHAIKH (23611780186)

GAYATRI DEORE (23611780237)

KHUSHI NIGAL (23611780235)

SALONI HANDE (23611780178)

FOR THE ACADEMIC YEAR

2025-2026

UNDER THE GUIDANCE OF

Prof. Y. N. JADHAV

SANDIP FOUNDATION

DEPARTMENT OF COMPUTER ENGINEERING

(NBA ACCREDITED)

SANDIP POLYTECHNIC, NASHIK

(An Affiliated Institute of Maharashtra State Board of Technical Education)

SANDIP FOUNDATION'S

SANDIP POLYTECHNIC, NASHIK

A/P: Mahiravani – 422213, Tal & Dist: Nashik

Website: http://www.sandipfoundation.org

E-mail: principal@sandippolytechnic.org Tel: (02594) 22571/72/73

Certificate

This is to certify that the project report entitled "**OBE TRACKING SYSTEM"** has been successfully completed by:

A. **Farheen Shaikh (23611780186)**
B. **Gayatri Deore (23611780237)**
C. **Khushi Nigal (23611780235)**
D. **Saloni Hande (23611780178)**

as partial fulfilment of Diploma course in Computer Engineering under the **Maharashtra State Board of Technical Education, Mumbai** during the academic year **2025-26**

The said work has been assessed by us and we are satisfied that the same is up to the standard envisaged for the level of the course. And that the said work may be presented to the external examiner.

Prof. Y. N. Jadhav
INTERNAL GUIDE

Prof. V. B. Ohol
HOD

EXTERNAL EXAMINER

Prof. P. M.Dharmadhikari
PRINCIPAL

ACKNOWLEDGEMENT

With deep sense of gratitude we would like to thanks all the people who have lit our path with their kind guidance. We are very grateful to these intellectuals who did their best to help during our project work.

It is our proud privilege to express deep sense of gratitude to, **Prof. P.M.Dharmardhikari, Principal** of Sandip Polytechnic, Nashik, for his comments and kind permission to complete this project. We remain indebted to **Prof . V. B Ohal, H.O.D Computer Engineering Department** for their timely suggestion and valuable guidance.

The special gratitude goes my guide **Prof. Y. N. Jadhav** and staff members, technical staff members of Computer Engineering Department for their expensive, excellent and precious guidance in completion of this work. We thank to all the colleagues for their appreciable help for our working project.

With various industry owners or lab technicians to help, it has been our endeavor to throughout our work to cover the entire project work.

We are also thankful to our parents who providing their wishful support for our project completion successfully.

And lastly we thanks to our all friends and the people who are directly or indirectly related to our project work.

You're sincerely,

FARHEEN SHAIKH

GAYATRI DEORE

KHUSHI NIGAL

SALONI HANDE

SPONSORSHIP LETTER

SANDIP FOUNDATION

SANDIP FOUNDATION'S

SANDIP POLYTECHNIC

Mahiravani, Trimbak Road, Tal. & Dist. Nashik-422213, Maharashtra, India.

Date:- 09/08/2026

Sponsorship Letter

Whom so ever it may concern

With reference to the meeting and presentation of final year project given by following students of third year diploma Computer Engineering, Sandip Polytechnic, Nashik, we have decided to offer a sponsorship to the said project.

Name of Students	Title of the Project
Saloni Hande	OBE Tracking System
Farheen Shaikh	
Khushi Nigal	
Gayatri Deore	

We wish all the best for their project and future.

Seal & Sign of company

HOD
Computer Engg. Department
Sandip Polytech...

PROJECT COMPETITION

SANDIP FOUNDATION'S

SANDIP POLYTECHNIC, NASHIK

State-Level Technical Events

techFEST 2K26

CERTIFICATE

This is to certify that Mr. / Miss. SHAIKH FARHEEN HUSSAIN MOHAMMAD

Program TYCO of Institute Sandip Polytechnic, Nashik

has Secured ~~1st / 2nd position~~ / Participated / Co-ordinated in Project Competition

event conducted on 13/03/2026, we wish him / her all the best for future.

Event Coordinator | Vice Principal | Prof. P. M. Dharmadhikari, Principal

SANDIP FOUNDATION'S

SANDIP POLYTECHNIC, NASHIK

State-Level Technical Events

techFEST 2K26

CERTIFICATE

This is to certify that Mr. / Miss. DEORE GAYATRI RAMDAS

Program TYCO of Institute Sandip Polytechnic, Nashik

has Secured ~~1st / 2nd position~~ / Participated / Co-ordinated in Project Competition

event conducted on 13/03/2026, we wish him / her all the best for future.

Event Coordinator | Vice Principal | Prof. P. M. Dharmadhikari, Principal


SANDIP FOUNDATION'S
SANDIP POLYTECHNIC, NASHIK
State-Level Technical Events
techFEST 2K26
CERTIFICATE
This is to certify that Mr. / Miss. NIGAL KHUSHI DNYANESHWAR
Program TYCO of Institute Sandip Polytechnic, Nashik
has Secured 1st / 2nd position / Participated / Co-ordinated in Project Competition
event conducted on 13/03/2026, we wish him / her all the best for future.
Event Coordinator
Vice Principal
Prof. P. M. Dharmadhikari
Principal
SANDIP FOUNDATION
PARAM ELECTROINFO
ROHNY


SANDIP FOUNDATION'S
SANDIP POLYTECHNIC, NASHIK
State-Level Technical Events
techFEST 2K26
CERTIFICATE
This is to certify that Mr. / Miss. HANDE SALONI PRABHAKAR
Program TYCO of Institute Sandip Polytechnic, Nashik
has Secured 1st / 2nd position / Participated / Co-ordinated in Project Competition
event conducted on 13/03/2026, we wish him / her all the best for future.
Event Coordinator
Vice Principal
Prof. P. M. Dharmadhikari
Principal
SANDIP FOUNDATION
PARAM ELECTROINFO
ROHNY

Krantiveer Vasantrao Narayanrao Naik Shikshan Prasarak Sanstha's

LOKNETE GOPINATHJI MUNDE INSTITUTE OF ENGINEERING EDUCATION & RESEARCH, NASHIK

(Diploma | Degree | MBA)

Dongre Vasatigruha Ground, Canada Corner, Sharanpur Road, Nashik - 422 002.

CERTIFICATE

This is to certify that

Mr./Ms. Farheen Shaikh *from* Sandip

Polytechnic *has participated / worked as Volunteer in* Project competition

State level competition during **"LoGMIEER Techfest 2k26"** *organized by Loknete Gopinathji Munde Institute of Engineering Education and Research on 27th February, 2026.*

Prof. P. S. Londhe (Event Co-ordinator)

Prof. S. S. Deshmukh (Event Co-ordinator)

Dr. K. V. Chandratre Principal

Krantiveer Vasantrao Narayanrao Naik Shikshan Prasarak Sanstha's

LOKNETE GOPINATHJI MUNDE INSTITUTE OF ENGINEERING EDUCATION & RESEARCH, NASHIK

(Diploma | Degree | MBA)

Dongre Vasatigruha Ground, Canada Corner, Sharanpur Road, Nashik - 422 002.

CERTIFICATE

This is to certify that

Mr./Ms. Gayatri Deore *from* Sandip

Polytechnic *has participated / worked as Volunteer in* Project competition

State level competition during **"LoGMIEER Techfest 2k26"** *organized by Loknete Gopinathji Munde Institute of Engineering Education and Research on 27th February, 2026.*

Prof. P. S. Londhe (Event Co-ordinator)

Prof. S. S. Deshmukh (Event Co-ordinator)

Dr. K. V. Chandratre Principal

Krantiveer Vasantrao Narayanrao Naik Shikshan Prasarak Sanstha's

LOKNETE GOPINATHJI MUNDE INSTITUTE OF ENGINEERING EDUCATION & RESEARCH, NASHIK

(Diploma | Degree | MBA)

Dongre Vasatigruha Ground, Canada Corner, Sharanpur Road, Nashik - 422 002.

CERTIFICATE

This is to certify that

Mr./Ms. Khushi Nigal *from* Sandip

Polytechnic *has participated / worked as Volunteer in* Project competition

State level competition during **"LoGMIEER Techfest 2k26"** *organized by Loknete Gopinathji Munde Institute of Engineering Education and Research on 27th February, 2026.*

Prof. P. S. Londhe	Prof. S. S. Deshmukh	Dr. K. V. Chandratre
(Event Co-ordinator)	(Event Co-ordinator)	Principal

Krantiveer Vasantrao Narayanrao Naik Shikshan Prasarak Sanstha's

LOKNETE GOPINATHJI MUNDE INSTITUTE OF ENGINEERING EDUCATION & RESEARCH, NASHIK

(Diploma | Degree | MBA)

Dongre Vasatigruha Ground, Canada Corner, Sharanpur Road, Nashik - 422 002.

CERTIFICATE

This is to certify that

Mr./Ms. Saloni Nande *from* Sandip

Polytechnic *has participated / worked as Volunteer in* Project Competition

State level competition during **"LoGMIEER Techfest 2k26"** *organized by Loknete Gopinathji Munde Institute of Engineering Education and Research on 27th February, 2026.*

Prof. P. S. Londhe	Prof. S. S. Deshmukh	Dr. K. V. Chandratre
(Event Co-ordinator)	(Event Co-ordinator)	Principal

Guru Gobind Singh Foundation's

Guru Gobind Singh Polytechnic Nashik

Institution's Innovation Council(IIC)-IC202221488

TECHNOBRAIN 2K26

CERTIFICATE

OF PARTICIPATION / ACHIEVEMENT

This certificate is proudly presented to:

Mast. / Ms. Farheen Shaikh

of Sandip Polytechnic **Institute**; studying in **First Year / Second Year / Third Year** Computer Engineering Branch has successfully **participated / secured First / Second / Third** rank in the event OBE tracking system under **TECHNOBRAIN 2K26.** This Technical Event is organized by **Guru Gobind Singh Polytechnic, Nashik,** held on **March 14, 2026.**

We appreciate the enthusiasm, technical skills and innovative spirit displayed during the event. We wish you great success in your future andeavors.

Prof. M. A. Borade	Prof. S. R. Upasani	K. K. S. Birdi	S. H. H. Anand	S. S. S. Chhabra
President, IIC	Principal, GGSP	Secrstary, GGSF	V. President, GGSF	President, GGSF

Event Sponsors

Guru Gobind Singh Foundation's

Guru Gobind Singh Polytechnic Nashik

Institution's Innovation Council(IIC)-IC202221488

TECHNOBRAIN 2K26

CERTIFICATE

OF PARTICIPATION / ACHIEVEMENT

This certificate is proudly presented to:

Mast. / Ms. Crayami Deore

of Sandip Polytechnic **Institute;** studying in **First Year / Second Year / Third Year** Computer Engineering Branch has successfully **participated / secured First / Second / Third** rank in the event OBE Tracking System under **TECHNOBRAIN 2K26.** This Technical Event is organized by **Guru Gobind Singh Polytechnic, Nashik,** held on **March 14, 2026.**

We appreciate the enthusiasm, technical skills and innovative spirit displayed during the event. We wish you great success in your future endeavors.

Prof. M. A. Borade	Prof. R. R. Upasani	S. K. S. Birdi	S. H. S. Anand	S. B. S. Chhabra
President, IIC	Principal, GGSP	Secretary, GGSF	V. President, GGSF	President, GGSF

Event Sponsors

Kumar Gardens · BHOGE ENTERPRISES · ANIRUDDHA VENTURES · Bonzer · BASALT · SM · IETE · SAYKHEDKAR · Sai Sports · NEW GANESH ART · ELECTRO MAGNETIC · r3sys · PRIME FOCUS ACADEMY · PANDURANG CORRECT WIRE ENTERPRISES · iCAD · SSPI&RC PVT.LTD. NASHIK · SAURABH STATIONARY NASHIK · SOFTCROWD TECHNOLOGIES · 3D shikshan · 24/7 Server Support · RAJBHOJ ELECTRICALS · SIRRA Fab

Guru Gobind Singh Foundation's

Guru Gobind Singh Polytechnic Nashik

Institution's Innovation Council(IIC)-IC202221488

TECHNOBRAIN 2K26

CERTIFICATE

OF PARTICIPATION / ACHIEVEMENT

This certificate is proudly presented to:

Mast. / Ms. Khushi Nigal

of Sandip Polytechnic **Institute;** studying in ~~First Year~~ / ~~Second Year~~ / **Third Year** Computer Engineering Branch has successfully **participated** / ~~secured~~ ~~First~~ / ~~Second~~ / ~~Third~~ rank in the event OBE tracking system under **TECHNOBRAIN 2K26.** This Technical Event is organized by **Guru Gobind Singh Polytechnic, Nashik,** held on **March 14, 2026.**

We appreciate the enthusiasm, technical skills and innovative spirit displayed during the event. We wish you great success in your future endeavors.

Prof. M. A. Borade	Prof. S. R. Upasani	S. K. S. Birdi	S. H. S. Anand	S. B. S. Chhabra
President, IIC	Principal, GGSP	Secretary, GGSF	V. President, GGSF	President, GGSF

Event Sponsors

Guru Gobind Singh Foundation's

Guru Gobind Singh Polytechnic Nashik

Institution's Innovation Council(IIC)-IC202221488

TECHNOBRAIN 2K26

CERTIFICATE

OF PARTICIPATION / ACHIEVEMENT

This certificate is proudly presented to:

Mast. / Ms. Saloni Hande

of Sandip Polytechnic **Institute;** studying in ~~First Year / Second Year /~~ Third Year Computer Engineering Branch has successfully **participated / secured** ~~**First / Second / Third**~~ rank in the event OBE tracking System under **TECHNOBRAIN 2K26.** This Technical Event is organized by **Guru Gobind Singh Polytechnic, Nashik**, hold on **March 14, 2026.**

We appreciate the enthusisam, technical skills and innovative spirit displayed during the event. We wish you great success in your future endeovors.

Prof. M. A. Borade	Prof. S. R. Upasani	K. K.S. Birdi	S. H. S. Anand	S. B. B. Chhabra
President, IIC	Principal, GSSP	Secretary, GBSF	V. President, GGSF	President, GGSF

Event Sponsors

K K WAGH EDUCATION SOCIETY'S

K K WAGH POLYTECHNIC, NASHIK

DEPARTMENT OF INFORMATION TECHNOLOGY

Chanakya Vedh 2026

CERTIFICATE

THIS IS TO CERTIFY THAT

Mr./ Miss.....Shaikh....Farheen....Hussain...Mohammad.....

from ....Sandip...Polytechnic.,..Nashik.....

has been awarded the ~~First~~/ Second/ ~~Third prize~~ / ~~Participated~~ in **Project Competition** under State level Technical Event **"Chanakya Vedh 2026"** held at **K K Wagh Polytechnic, Nashik** on 17th March 2026.

Mrs. D. D. PAWAR	Ms. M. S. KARANDE	PROF. P. T. KADAVE
Coordinator	HOD	PRINCIPAL

K K WAGH EDUCATION SOCIETY'S

K K WAGH POLYTECHNIC, NASHIK

DEPARTMENT OF INFORMATION TECHNOLOGY

Chanakya Vedh 2026

CERTIFICATE

THIS IS TO CERTIFY THAT

Mr./ Miss....Deore.....Gayatri....Ramdas.....

from .....Sandip....Polytechnic.,...Nashik.....

has been awarded the ~~First~~/ Second/ ~~Third prize~~ / ~~Participated~~ in **Project Competition** under State level Technical Event **"Chanakya Vedh 2026"** held at **K K Wagh Polytechnic, Nashik** on 17th March 2026.

Mrs. D. D. PAWAR	Ms. M. S. KARANDE	PROF. P. T. KADAVE
Coordinator	HOD	PRINCIPAL

K K WAGH EDUCATION SOCIETY'S

K K WAGH POLYTECHNIC, NASHIK

DEPARTMENT OF INFORMATION TECHNOLOGY

Chanakya Vedh 2026

CERTIFICATE

THIS IS TO CERTIFY THAT

Mr./ Miss.....Nigal....Khushi....Dnyaneshwar.....

from .....Sandip....Polytechnic,...Nashik.....

has been awarded the ~~First~~/ Second/ ~~Third prize~~ / ~~Participated~~ in **Project Competition** under State level Technical Event **"Chanakya Vedh 2026"** held at **K K Wagh Polytechnic, Nashik** on 17th March 2026.

Mrs. D. D. PAWAR	Ms. M. S. KARANDE	PROF. P. T. KADAVE
Coordinator	HOD	PRINCIPAL

K K WAGH EDUCATION SOCIETY'S

K K WAGH POLYTECHNIC, NASHIK

DEPARTMENT OF INFORMATION TECHNOLOGY

Chanakya Vedh 2026

CERTIFICATE

THIS IS TO CERTIFY THAT

Mr./ Miss.....Hande.....Salani....Prabhakar.....

from ....Sandip....Polytechnic,...Nashik.....

has been awarded the ~~First~~/ Second/ ~~Third prize~~ / ~~Participated~~ in **Project Competition** under State level Technical Event **"Chanakya Vedh 2026"** held at **K K Wagh Polytechnic, Nashik** on 17th March 2026.

Mrs. D. D. PAWAR	Ms. M. S. KARANDE	PROF. P. T. KADAVE
Coordinator	HOD	PRINCIPAL

PAPER PUBLICATION

International Journal of Scientific Development and Research
IJSDR.ORG | ISSN : 2455-2631
An International Open Access, Peer-reviewed, Refereed Journal

Certificate of Publication

The Board of
International Journal of Scientific Development and Research
Is hereby awarding certificate to
Shaikh Farheen
In recognition of the publication of the paper entitled
A Role-Based Web System for Outcome Based Education Attainment and Reporting
Published in Volume 11 Issue 1, January-2026, | Impact Factor: 9.15 by Google Scholar
Co-Authors - Hande Saloni, Nigal Khushi, Deore Gayatri, Mr. Y. N. Jadhav

Paper ID - IJSDR2601161
Registration ID - 307056

Editor-In Chief

IJSDR | ISSN : 2455-2631

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator
IJSDR - International Journal of Scientific Development and Research | ESTD: 2016

www.ijsdr.org | editor@ijsdr.org

International Journal of Scientific Development and Research
IJSDR.ORG | ISSN : 2455-2631
An International Open Access, Peer-reviewed, Refereed Journal

Certificate of Publication

The Board of
International Journal of Scientific Development and Research
Is hereby awarding certificate to
Deore Gayatri
In recognition of the publication of the paper entitled
A Role-Based Web System for Outcome Based Education Attainment and Reporting
Published in Volume 11 Issue 1, January-2026, | Impact Factor: 9.15 by Google Scholar
Co-Authors - Hande Saloni, Shaikh Farheen, Nigal Khushi, Mr. Y. N. Jadhav

Paper ID - IJSDR2601161
Registration ID - 307056

Editor-In Chief

IJSDR | ISSN : 2455-2631

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator
IJSDR - International Journal of Scientific Development and Research | ESTD: 2016

www.ijsdr.org | editor@ijsdr.org

International Journal of Scientific Development and Research
IJSDR.ORG | ISSN : 2455-2631
An International Open Access, Peer-reviewed, Refereed Journal

Certificate of Publication

The Board of
International Journal of Scientific Development and Research
Is hereby awarding certificate to

Nigal Khushi

In recognition of the publication of the paper entitled
A Role-Based Web System for Outcome Based Education Attainment and Reporting
Published in Volume 11 Issue 1, January-2026, | Impact Factor: 9.15 by Google Scholar
Co-Authors - Hande Saloni, Shaikh Farheen, Deore Gayatri, Mr. Y. N. Jadhav

Paper ID - IJSDR2601161
Registration ID - 307056

Editor-In Chief

IJSDR | ISSN : 2455-2631

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator
IJSDR - International Journal of Scientific Development and Research | ESTD: 2016
www.ijsdr.org | editor@ijsdr.org

International Journal of Scientific Development and Research
IJSDR.ORG | ISSN : 2455-2631
An International Open Access, Peer-reviewed, Refereed Journal

Certificate of Publication

The Board of
International Journal of Scientific Development and Research
Is hereby awarding certificate to

Hande Saloni

In recognition of the publication of the paper entitled
A Role-Based Web System for Outcome Based Education Attainment and Reporting
Published in Volume 11 Issue 1, January-2026, | Impact Factor: 9.15 by Google Scholar
Co-Authors - Shaikh Farheen, Nigal Khushi, Deore Gayatri, Mr. Y. N. Jadhav

Paper ID - IJSDR2601161
Registration ID - 307056

Editor-In Chief

IJSDR | ISSN : 2455-2631

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator
IJSDR - International Journal of Scientific Development and Research | ESTD: 2016
www.ijsdr.org | editor@ijsdr.org

PAGE INDEX

TABLE INDEX

FIGURE INDEX

CHAPTER 0. ABSTRACT

Outcome *Based Education (OBE) is a mandatory framework for engineering institutions to ensure measurable learning outcomes and compliance with accreditation bodies such as NBA and NAAC. In practice, many institutions rely on manual or spreadsheet based processes for entering assessment data, calculating attainment, comparing targets, and preparing reports. These approaches are time consuming, error prone, and difficult to manage during audits.*

This project presents a web-based OBE Tracking System that supports faculty-defined CO–PO–PSO mappings and automates key processes including direct and indirect attainment calculation, target comparison, gap identification, outcome backtracking, and generation of accreditation-ready reports. Attainment is computed dynamically as soon as relevant assessment data or indirect survey responses are recorded in the system, enabling real-time monitoring of outcome achievement and facilitating timely corrective actions.

The system also integrates stress related feedback collected directly from students through secure survey links. A pilot stress survey was conducted to evaluate the relevance of stress analysis in an academic context, and the findings guided the design of the final stress survey module. Role based access control ensures structured data entry, verification, approval, and audit review. The proposed system reduces manual workload, improves accuracy, enhances audit readiness, and supports continuous improvement under the OBE framework.

Keywords: *Outcome Based Education, Attainment Calculation, CO PO PSO Mapping, Stress Analysis, Accreditation, Web Based System.*

CHAPTER 1. INTRODUCTION

The quality of technical education plays an important role in the development of skilled manpower for industry and society. Educational institutions are expected to ensure that students not only gain theoretical knowledge but also develop practical skills, problem-solving ability, and professional competence.

Traditional education systems mainly focused on syllabus completion and examination results. However, modern education requires students to develop analytical thinking, communication skills, teamwork, and ethical values. To address these requirements, Outcome Based Education (OBE) has emerged as an effective approach that focuses on what students are able to achieve after completing a course or program.

OBE ensures that learning outcomes are clearly defined, measurable, and aligned with industry expectations. It also promotes continuous improvement in teaching–learning processes by regularly analyzing student performance and identifying gaps.

1.1. Outcome Based Education (OBE) Framework

Outcome Based Education is an approach that emphasizes learning outcomes rather than teaching methods alone. In this system, the entire academic process is designed to ensure that students achieve specific skills and competencies.

The key components of OBE include:

- Program Educational Objectives (PEOs)
- Program Outcomes (POs)
- Program Specific Outcomes (PSOs)
- Course Outcomes (COs)
- CO–PO–PSO Mapping

Student performance is evaluated using direct assessment tools such as exams, assignments, and practical work, and indirect tools such as feedback surveys. The results are analyzed to improve teaching effectiveness and student learning.

1.2. Role of NBA in Quality Assurance

In India, the implementation and evaluation of Outcome Based Education is supported by the National Board of Accreditation (NBA).

The National Board of Accreditation (NBA) is an autonomous body established to assess and accredit technical and professional education programs. It ensures that institutions follow structured academic processes and maintain quality standards.

NBA evaluates programs based on defined criteria such as curriculum, teaching–learning processes, student performance, faculty contribution, and continuous improvement. It promotes the adoption of OBE and ensures that institutions align their academic practices with national and international standards.

1.3. Need for Quality Assurance in Education

The need for structured quality assurance arises from increasing industry expectations. Employers require graduates who are capable of applying knowledge to real-world problems.

Traditional evaluation methods based only on marks are not sufficient to measure student competence. A systematic approach is required to track learning outcomes, evaluate performance, and improve academic processes.

Quality assurance frameworks help institutions:

- Define clear learning objectives
- Measure student performance effectively
- Identify gaps in learning
- Implement continuous improvement strategies

1.4. Benefits of Outcome Based Education (OBE)

Outcome Based Education provides several benefits to students, faculty, and institutions.

Students gain practical knowledge and skills required for their careers. Faculty members can improve teaching strategies based on performance analysis. Institutions benefit from better academic planning, structured documentation, and improved credibility.

OBE also ensures transparency and accountability in academic processes.

1.5. Key Evaluation Parameters (NBA Perspective)

To ensure quality education, accreditation bodies such as NBA evaluate academic programs based on multiple parameters, including:

- Vision, Mission and Program Educational Objectives.
- Program Curriculum and Teaching–Learning Processes.
- Course Outcomes and Program Outcomes.
- Students' Performance.
- Faculty Information and Contributions.
- Facilities and Technical Support.
- Continuous Improvement.
- First Year Academics.
- Student Support Systems.
- Governance, Institutional Support and Financial Resources.

CHAPTER 2. INTRODUCTION TO PROPOSED SYSTEM

2.1. Concept of Outcome Based Education

OBE is a systematic academic framework that focuses on defining measurable learning outcomes that students must achieve at the end of a course or program. Unlike traditional education systems that emphasize syllabus completion and examination scores, OBE concentrates on the actual knowledge, skills, and competencies students gain during their academic journey.

In OBE, the entire teaching-learning process is designed backward. First, the desired outcomes are defined. Then curriculum design, teaching methods, and assessment strategies are aligned to ensure those outcomes are achieved effectively.

The main components of OBE include:

- ***Course Outcomes (COs):*** These describe what students are expected to learn in a specific subject.
- ***Program Outcomes (POs):*** These represent broader abilities such as problem-solving, teamwork, communication, and ethical responsibility that students should develop by the end of the program.
- ***Program Specific Outcomes (PSOs):*** These define discipline-specific skills relevant to a particular field of study.

The attainment of these outcomes is measured through both direct and indirect assessment methods. Direct assessments include exams, assignments, and practical evaluations, while indirect assessments involve surveys and feedback mechanisms.

OBE promotes continuous improvement by analyzing attainment levels and identifying gaps between expected and actual performance. This structured evaluation approach ensures quality education and accountability within institutions.

2.2. Existing Practices in OBE Implementation

Currently, many educational institutions implement OBE using manual processes and spreadsheet-based tools such as Microsoft Excel. Faculty members perform CO–PO mapping manually and calculate attainment using predefined formulas.

The common steps followed in existing systems include:

- Defining COs, POs, and PSOs.
- Mapping COs to POs and PSOs using correlation levels.
- Entering student marks into spreadsheets.
- Conducting course exit surveys for course indirect attainment.
- Calculating direct attainment using pre-defined formulas.
- Conduction feedback surveys for indirect attainment.
- Calculating indirect attainment based on feedback responses using pre-defined formulas.
- Calculating overall PO/PSO attainment using pre-defined formula.
- Preparing reports for accreditation documentation.

Although spreadsheets offer flexibility, they have several limitations. Data is often stored in multiple files across departments, leading to inconsistency. Minor errors in formulas can result in incorrect attainment calculations. Additionally, generating consolidated institutional reports requires manual compilation from various sources.

Version control issues also arise when multiple faculty members edit the same files. Maintaining historical records semester-wise becomes difficult. These challenges increase faculty workload and reduce efficiency.

Therefore, existing practices lack automation, centralized storage, and structured monitoring mechanisms.

2.3. Objective of The System

The proposed OBE Tracking System is developed to overcome the limitations of manual and spreadsheet-based approaches. The system aims to provide a centralized and automated platform for managing all OBE-related processes.

The primary objectives of the system are:

- To manage faculty-defined CO–PO–PSO mapping.
- To calculate direct attainment based on student performance automatically.
- To integrate indirect attainment using survey responses.
- To generate consolidated attainment reports in real-time.
- To provide role-based access control for Admin, HOD, Faculty, Coordinator and Auditor.
- To ensure secure and centralized database storage.

- To maintain historical data for accreditation audits.
- To support continuous improvement strategies through automated gap analysis.

By achieving these objectives, the system reduces manual effort, enhances accuracy, and ensures transparency in academic evaluation processes.

2.4. Scope of The System

The scope of the proposed OBE Tracking System includes the design and development of a web-based application that manages academic outcome tracking within an institution.

The system covers the following functional areas:

- CO–PO–PSO mapping management.
- Student marks entry and storage.
- Direct attainment calculation.
- Indirect attainment integration through surveys.
- Weighted average computation of final PO attainment.
- Gap analysis and improvement tracking.
- Generation of accreditation-ready reports.
- Role-based access management.
- Secure database maintenance.

The system can be implemented at the departmental level or across the entire institution. It supports multiple academic years and maintains semester-wise records for comparison and improvement analysis.

In the future, the system can be extended with advanced features such as real-time dashboards, graphical performance visualization, AI-based predictive analytics, and integrated stress monitoring surveys for faculty and students.

Overall, the proposed system provides a structured, efficient, and scalable solution for implementing Outcome-Based Education in compliance with accreditation requirement.

CHAPTER 3. PROBLEM STATEMENT

3.1. Background of The Problem

In recent years, Outcome-Based Education (OBE) has become an essential academic framework in engineering and technical institutions. Accreditation bodies such as NBA and NAAC emphasize measurable learning outcomes and continuous improvement in teaching–learning processes. Under OBE, institutions must clearly define Course Outcomes (COs), Program Outcomes (POs), and Program Specific Outcomes (PSOs), and measure the extent to which these outcomes are attained.

The core principle of OBE is that education should be outcome-driven rather than content-driven. This means that the success of a course or program is evaluated not merely by syllabus completion, but by the measurable achievement of defined learning outcomes by students.

To implement OBE effectively, institutions are required to perform the following activities:

- Define clear and measurable Course Outcomes (COs) for each subject.
- Map COs to relevant Program Outcomes (POs) and Program Specific Outcomes (PSOs).
- Conduct internal and external assessments linked to COs.
- Calculate direct and indirect attainment levels.
- Analyze gaps and plan corrective actions.
- Prepare structured documentation for accreditation.

While the theoretical framework of OBE is well defined, its practical implementation in many institutions remains largely manual or semi-automated. Most departments rely heavily on spreadsheet tools and manual documentation, which creates several operational challenges.

As the number of students, courses, faculty members, and accreditation requirements increases, managing OBE processes manually becomes complex, time-consuming, and error-prone.

3.2. Existing System and Current Practices

Currently, OBE implementation in many colleges follows a decentralized and semi-automated workflow. Faculty members maintain separate Excel sheets for each subject and semester. These sheets typically include:

- CO definitions.
- CO–PO mapping tables.

- Internal assessment marks.
- External examination marks.
- Attainment calculation formulas.

The general process followed is:

- Creation of CO–PO–PSO mapping matrices.
- Conducting internal assessments (Unit Tests, Assignments, Practical exams).
- Entry of marks in Excel sheets.
- Manual calculation of CO attainment percentages.
- Aggregation of results to compute PO attainment.
- Preparation of reports for department review.

In many case, the calculation of direct attainment follows a weighted formula such as:

DirectCO = (0.4 × Internal Assessment) + (0.6 × External Examination)

Similarly, overall PO attainment may include both direct and indirect components:

FinalPO = (0.8 × DirectPO) + (0.2 × IndirectPO)

Although these formulas are conceptually simple, implementing them manually across multiple courses and hundreds of students increases the possibility of calculation errors. At the end of the semester, faculty members submit individual files to the Head of Department (HOD), who then consolidates all data manually to prepare department-level reports.

While this system fulfills minimum compliance requirements, it lacks efficiency, automation, and real-time monitoring capabilities.

3.3. Limitations of The Existing System

The manual and spreadsheet-based approach presents several limitations:

3.3.1. Data Inconsistency:

Different faculty members may design Excel sheets using different formats, naming conventions, and formulas. This results in:

- Lack of uniformity in CO–PO mapping.
- Variations in attainment calculation logic.
- Difficulty in comparing results across departments.

The absence of standardized templates leads to inconsistent academic data.

3.3.2. High Probability of Calculation Errors:

Manual formula entry in spreadsheets increases the risk of:

- Incorrect cell references.
- Formula misplacement.
- Omission of certain students' data.
- Errors in weighted calculations

Even minor mistakes can significantly affect attainment percentages, which may impact accreditation evaluation.

3.3.3. Version Control and Data Duplication Issues:

Multiple copies of Excel files are often shared via:

- Email.
- Pen drives.
- Cloud storage platforms.

This creates problems such as:

- Confusion regarding latest version.
- Accidental deletion of updated files.
- Data duplication.
- Difficulty in tracking historical records.

There is no centralized database to ensure single-source data integrity

3.3.4. Lack of Role-Based Access Control:

In the existing system, there is no structured mechanism to restrict access to academic data based on user roles and responsibilities. Excel files are often shared among faculty members, HODs, and coordinators without any defined access permissions.

This leads to several issues:

- Unauthorized access to sensitive data such as student marks and attainment reports.
- No restriction on editing, allowing any user to modify critical data.
- Lack of accountability, as changes made by users cannot be tracked.
- Risk of accidental data modification or deletion.
- No clear separation of responsibilities between faculty, HODs, and administrators.

In addition, file sharing through email, pen drives, or cloud platforms further increases security risks and chances of data misuse.

The absence of role-based access control affects data security, integrity, and reliability, making the system unsuitable for handling critical academic information.

3.3.5. Increased Faculty Workload:

Faculty members spend considerable time on:

- Manual data entry.
- Repetitive calculations.
- Formatting reports.
- Cross-verification of marks.
- Preparing documentation during audits.

Instead of focusing on teaching and academic improvement, significant time is consumed by administrative tasks.

3.3.6. Limited Real-Time Monitoring:

In the existing system:

- Attainment results are available only after manual calculation.
- HODs cannot monitor course progress in real time.
- There is no dashboard for visualizing trends.

Decision-making becomes reactive rather than proactive

3.3.7. Stress During Accreditation Audits:

During NBA or NAAC inspections:

- Departments must provide multiple years of attainment data.
- Faculty members must produce evidence of calculations.
- Reports must be presented in structured format.

Due to scattered data storage, collecting required documents becomes stressful and time-consuming.

3.4. User - Identified Challenges

Through discussions with faculty members, HODs, and academic coordinators, the following practical challenges were identified:

- Difficulty in maintaining uniform data across semesters.
- Challenges in combining multiple faculty reports.
- Absence of automatic gap analysis.
- Lack of centralized performance tracking.
- Inability to quickly generate accreditation-ready reports.
- Increased stress due to repetitive documentation.

Additionally, indirect attainment data from surveys such as Course Exit Surveys, Curricular Activity Feedback, Activity Resource Person Feedback, Alumni Feedback, and Program Exit Survey are often collected manually and stored separately, making integration with direct attainment difficult.

The absence of an integrated platform prevents long-term academic performance analysis and trend prediction.

3.5. Need for the Proposed System

Considering the limitations of the existing system, there is a clear need for:

- A centralized web-based platform.
- Automated attainment calculations.
- Standardized CO–PO mapping templates.
- Role-based authentication and authorization.
- Secure database storage.
- Real-time dashboards for monitoring.
- Automatic report generation.

An integrated system can significantly reduce manual effort and improve data reliability.

3.6. Problem Statement

Based on the detailed analysis of current practices and challenges, the core problem can be formally stated as:

The existing manual and semi-automated methods used for implementing Outcome-Based

Education lack standardization, automation, centralized control, and secure data management. This results in inconsistent calculations, increased faculty workload, inefficient reporting processes, data security risks, and difficulty in meeting accreditation requirements effectively. Therefore, there is a strong requirement for a centralized, automated, secure, and user- friendly OBE Tracking System that streamlines academic data management, reduces errors, and supports continuous quality improvement in educational institutions.

CHAPTER 4. PROPOSED SYSTEM

4.1. Objectives of the Proposed System

The primary objectives of the OBE Tracking System are:

- To manage CO, PO, and PSO mapping processes.
- To eliminate manual errors in attainment calculation.
- To provide centralized storage of academic and assessment data.
- To generate standardized accreditation-ready reports.
- To implement secure role-based access control.
- To provide real-time dashboards for academic monitoring.
- To integrate direct and indirect attainment analysis.
- To reduce documentation-related stress on faculty members.

The system aims to enhance transparency, efficiency, and data accuracy in academic outcome measurement.

4.2. Overview of the Proposed System

The proposed system is divided into backend modules and frontend modules.

Backend Modules:

- User Management.
- Academic Configuration.
- CIS Master.
- Assessments.
- Attainment Calculation.
- Indirect Attainment.
- Surveys.
- Reports.
- Teaching Plan.
- Audit.
- Bulk Upload.
- Dashboard APIs.

Frontend Modules:

- Admin User.
- HOD User.
- Faculty User.
- Student User.
- Auditor User.
- Common Components.

Each module performs specific responsibilities while interacting with other modules to ensure smooth data flow.

4.3. Functional Features of the Proposed System

4.3.1. User Management and Authentication:

The system provides secure login functionality for different user roles:

- Administrator.
- Head of Department (HOD).
- NBA Criterion 3 Coordinator
- Faculty.
- Auditor.

Each user role has specific permissions and access rights. For example:

- Admin can configure academic structures.
- HOD can monitor departmental attainment.
- NBA Criterion 3 Coordinator monitor departmental Criterion 3 processes.
- Faculty can enter marks and manage courses.
- Auditor can view reports and give remarks.
- Students can only respond to surveys, but cannot login.

This ensures secure and controlled data access.

4.3.2. Academic Configuration:

The system allows structured configuration of:

- Departments.
- Schemes.

- Batches.
- Academic Years.
- Semesters.
- Courses.
- Faculty Assignments.

All academic data is stored in a centralized database, ensuring consistency across departments.

4.3.3. Assessment Management:

The assessment module manages:

- Internal assessments (Unit Test, Assignment, Practical).
- External examination marks.
- Question-wise / Assessment-wise CO mapping.
- Student marks entry.

Faculty can directly enter marks through secure interface. The system validates data before saving, reducing errors.

4.3.4. Direct Attainment Calculation:

Direct attainment is calculated automatically using predefined weighted formulas. For example:

DirectCO = (0.4 × InternalCO) + (0.6 × ExternalCO)

The system computes attainment for each CO based on student performance thresholds.

PO attainment is derived using CO–PO mapping relationships. This automation eliminates manual spreadsheet dependency.

4.3.5. Indirect Attainment Calculation:

Indirect attainment is calculated using survey feedback data such as:

- Course Exit Survey (Course level).
- Curricular Activity Feedback (Expert Lecture, Industry Visit, etc).
- Activity Resource Person Feedback.
- Alumni Feedback.
- Program Exit Survey.

Final attainment calculation may follow:

FinalPO = (0.8 × DirectPO) + (0.2 × IndirectPO)

The system automatically converts survey responses into numeric scores and integrates them with direct attainment.

4.3.6. Survey Management:

The system includes dynamic survey creation features:

- Survey Question management.
- Automatic response collection.
- Survey result calculation.

Survey results are directly linked to indirect attainment calculations.

4.3.7. Stress Monitoring Module:

A unique feature of the proposed system is the Student Stress Monitoring Module. This module:

- Conducts stress-related questionnaires.
- Evaluates student responses.
- Generates stress-level reports.
- Provides recommendations.

This helps in monitoring student well-being along with academic performance.

4.3.8. Teaching Plan Management:

Faculty can:

- Create teaching plans.
- Monitor topic coverage.

This ensures alignment between teaching activities and CO attainment.

4.3.9. Report Generation:

The system automatically generates:

- CIS Reports (Direct Attainment).
- Indirect Attainment Reports.

- PO / PSO Attainment Reports.
- Stress Analysis Reports.

Reports can be exported in:

- Excel format.

These reports follow accreditation documentation standards.

4.3.10. Dashboard and Data Visualization:

The system provides graphical dashboards for:

- CO attainment trends.
- PO / PSO attainment trends.
- Survey status.

This helps management in quick decision-making.

4.3.11. Audit and Logging:

The audit module:

- Tracks user activity.
- Maintains change history.
- Logs data modifications.
- Maintains access logs.

This improves system security and accountability.

4.3.12. Bulk Data Upload:

To reduce repetitive data entry, the system allows:

- Student list upload.
- Marks upload.
- Teaching plan upload.

Excel templates are provided for structured uploading.

4.4. System Architecture

The proposed system follows a three-tier architecture:

- ***Presentation Layer:*** Frontend developed using client-side framework (React.js +

Bootstrap).

- ***Application Layer:*** Backend developed using server-side framework (Django / REST API).
- ***Database Layer:*** Centralized relational database for secure data storage (PostgreSQL).

This layered approach ensures scalability, maintainability, and modular development.

4.5. Feasibility Study

4.5.1. *Technical Feasibility:*

The system is technically feasible because:

- It uses widely available web technologies.
- Requires only basic server infrastructure.
- Can run on institutional networks.
- Does not require specialized hardware.

The attainment formulas and logic are simple to implement in backend code.

4.5.2. Economic Feasibility:

The proposed system is economically viable because:

- It uses open-source frameworks.
- No expensive licensing cost.
- Minimal hardware investment.
- Reduces paper and printing cost.
- Saves faculty time.

The long-term benefits justify the development cost.

4.5.3. Operational Feasibility:

The system is easy to operate because:

- User-friendly interface.
- Minimal training required.
- Role-based workflow matches academic structure.
- Can be gradually implemented.

Faculty can transition smoothly from Excel-based methods to the automated system.

4.6. Advantages of the Proposed System

- Eliminates manual calculation errors.
- Provides centralized data storage.
- Improves data consistency.
- Reduces faculty workload.
- Ensures secure role-based access.
- Generates instant reports.
- Supports accreditation processes.
- Enhances academic monitoring.

CHAPTER 5. SYSTEM ARCHITECTURE

5.1. Introduction

System Architecture defines the structural design and operational framework of the proposed Outcome Based Education (OBE) Tracking System. It describes how different components of the system interact with each other to ensure efficient processing, secure data handling, and structured academic outcome management.

The proposed OBE Tracking System is designed as a centralized, web-based application that automates the implementation of outcome-based education processes within an academic institution. The system integrates multiple user roles such as Admin, Faculty, Head of Department (HOD), Coordinator, and Auditor under a structured role-based access mechanism.

The system is developed using:

- **Backend Technology**: Django REST Framework (Python-based).
- **Frontend Technology:** HTML, CSS, Bootstrap.
- **Database:** SQLite (Development), PostgreSQL (Deployment).
- **Architecture Model:** Three-Tier Client–Server Architecture.

This architecture ensures separation of concerns, maintainability, scalability, and data security.

5.2. Architectural Model

The proposed system follows a Three-Tier Architecture, which divides the application into three independent layers:

- Presentation Layer (User Interface Layer).
- Application Layer (Business Logic Layer).
- Data Layer (Database Layer).

This layered approach ensures that modifications in one layer do not directly affect other layers, thereby improving system stability and maintainability.

5.3. Overall System Structure

The OBE Tracking System operates on a centralized server model where all users access the system through a web browser.

Basic Operational Flow:

User (Admin / Faculty / HOD / Coordinator / Auditor)

↓

Web Browser (Client Interface)

↓

Django Web Server (Application Processing)

↓

PostgreSQL Database (Data Storage)

↓

Processed Response Returned to User

Fig. No. 1. Basic Operational Flow

All requests are processed through the backend server, ensuring secure and controlled data access.

5.3.1. Presentation Layer (Frontend Architecture):

The Presentation Layer is responsible for user interaction and display of information. It is developed using HTML, CSS, and Bootstrap to ensure responsive and structured interface design.

Major Functional Interfaces Include:

- Login Page.
- Role-Based Dashboard.
- Course Outcome Entry.
- CO–PO–PSO Mapping Interface.
- Marks Entry Module.
- Attainment Calculation Reports.
- Stress Survey Module.

Bootstrap framework is used to:

- Provide responsive design.
- Ensure mobile compatibility.
- Maintain uniform styling.
- Create structured navigation.

- Improve user experience.

Each user role sees only the features permitted under their access level, ensuring security and clarity.

5.3.2. Application Layer (Django Backend Architecture):

The Application Layer is the core of the system where all processing, validation, and business logic execution occurs. It is implemented using the Django REST framework.

Django provides a structured MVT architecture consisting of:

- Models.
- Views.
- Templates.
- URL Routing.
- Authentication System

Business Logic Processing (The backend handles):

- Role-based authentication.
- Authorization checks.
- CO–PO–PSO mapping logic.
- Attainment level calculation.
- Report generation.
- Data validation.
- Approval workflows.
- Survey data analysis.

The attainment calculation logic is automated to reduce manual errors and ensure consistent computation across subjects and departments.

5.3.3. Role-Based Architecture (RBAC Model):

The system implements a Role-Based Access Control (RBAC) mechanism to ensure structured and secure operations.

- ***Admin:*** The Admin acts as the system controller. Responsibilities:
    - Create and manage user accounts.
    - Assign roles (Faculty, HOD, Coordinator, and Auditor).

- Manage departments and schemes.
- Monitor system activities.

The Admin has complete access to system configuration.

- ***Faculty:*** Faculty members are responsible for academic data entry and course-level outcome implementation. Responsibilities:
  - Define Course Outcomes (COs).
  - Map COs with POs and PSOs.
  - Enter internal and external assessment mark.
  - View attainment calculation results.
  - Submit reports for review.

  Faculty access is restricted to assigned subjects only.

- ***Head of Department (HOD):*** The HOD supervises outcome implementation at the departmental level. Responsibilities:
  - Review faculty-submitted data.
  - Verify CO–PO-PSO mappings.
  - Approve or reject attainment reports.
  - Monitor department performance.

  HOD ensures quality control before institutional consolidation.

- ***Coordinator:*** The Coordinator oversees departmental NBA Criterion 3 implementation. Responsibilities:
  - Verify mapping consistency across courses.
  - View attainment reports.
  - Monitor compliance with regulatory standards.

  The Coordinator ensures alignment with accreditation requirements such as NBA or NAAC standards.

- ***Auditor:*** The Auditor role ensures transparency and compliance. Responsibilities:
  - Access read-only reports.

- Review attainment records.
- Verify documentation.
- Inspect mapping structures.
- Evaluate compliance with academic policies.

The Auditor cannot modify any data, ensuring data integrity.

5.3.4. Data Layer Architecture (PostgreSQL Database):

The Data Layer consists of a centralized PostgreSQL database that stores all structured data.

Major Database Tables:

- User Table.
- Role Table.
- Department Table.
- Course Table.
- CO Table.
- PO Table.
- PSO Table.
- CO–PO–PSO Mapping Table.
- Marks Table.
- Attainment Table.
- Approval Status Table.
- Survey Response Table.

PostgreSQL is chosen because:

- Robust and highly reliable relational database system.
- Supports advanced features such as complex queries, indexing, and constraints.
- Scalable and suitable for handling large volumes of data and concurrent users.
- Seamless integration with Django using the built-in PostgreSQL adapter.

Django ORM ensures secure database interaction and prevents SQL injection vulnerabilities.

5.4. Data Flow Architecture

The system follows a structured data flow process:

- User logs into the system.

- Authentication verifies credentials and role.
- Faculty enters academic data.
- Backend validates inputs.
- Data is stored in the database.
- Attainment calculation is executed automatically.
- Reports are generated.
- HOD reviews and approves.
- Coordinator reviews Criterion 3 processes.
- Auditor views reports and give remarks.

This structured flow ensures accountability at every stage.

5.5. Security Architecture

Security is an integral part of the system architecture. Security mechanisms include:

- Django Authentication System.
- Role-Based Access Control.
- Encrypted Password Storage.
- Session Management.
- CSRF Protection.
- Form Validation.
- Restricted URL Access.

Sensitive modules such as report approval and mapping modification are accessible only to authorized roles.

5.6. Deployment Architecture

The system can be deployed in multiple environments depending on institutional requirements:

- ***Institutional server:*** Hosted on on-premises servers within the college or university network.
- ***Local development server:*** Suitable for development, testing, and demonstration purposes.
- ***Cloud-based server:*** Enables future scalability and remote access, reducing dependency on local infrastructure.

Being a web-based system, the platform requires only a web browser on client machines, with

no additional software installation needed. The system can be accessed seamlessly within the campus network, providing easy availability to faculty and administrators.

The application follows a single-service deployment model:

- The React frontend is built into static files that are served by Django's static file handling mechanism.
- Django backend exposes RESTful APIs to handle all business logic, including attainment calculations, survey processing, and report generation.
- The backend is served using Gunicorn, a reliable WSGI server, ensuring efficient request handling and concurrency.
- PostgreSQL database, hosted on Render, stores all academic and survey data, providing a robust and scalable storage solution.

This architecture ensures a clear separation between the frontend and backend, simplifies deployment, and minimizes hardware requirements. The system is compatible with standard institutional infrastructure and can be maintained easily without specialized resources.

5.7. Advantages of the Adopted Architecture

The architecture provides several advantages:

- Modular and scalable design.
- Secure multi-user access.
- Automated attainment calculations.
- Reduced manual errors.
- Centralized data management.
- Accreditation-ready reporting.
- Transparency and audit tracking.
- Easy maintenance and future enhancement.

CHAPTER 6. METHODOLOGY

This chapter explains the overall procedure adopted for developing the OBE Tracking System. The methodology focuses on creating a structured, reliable, and automated approach for managing academic outcomes. The system is designed to support institutions in implementing Outcome Based Education effectively and systematically. The entire development process was carried out in planned stages to ensure proper design, accurate calculation, and secure data handling. Special attention was given to automation, transparency, and continuous academic improvement.

6.1. Development Strategy

The system was developed using a step-by-step modular strategy. Instead of building the complete application at once, the project was divided into smaller functional parts. Each part was carefully designed, coded, and tested before combining it with other modules.

The major functional sections of the system include:

- User Account Management.
- Academic Structure Configuration.
- Outcome Mapping.
- Assessment Data Entry.
- Attainment Processing.
- Report Generation.
- Student Stress Analysis Module.

This approach improved system reliability and allowed early detection and correction of error.

6.2. Structured Implementation of OBE

The OBE process inside the system follows a logical and organized sequence.

- ***Phase 1: Outcome Definition***

  Initially, the academic outcomes are defined at different levels. These include:

  - Program Outcomes (POs).
  - Program Specific Outcomes (PSOs).
  - Course Outcomes (COs).

  Authorized users enter these outcomes into the system database. These defined outcomes

form the base for further evaluation.

- ***Phase 2: Outcome Mapping***

  Each Course Outcome is connected with relevant Program Outcomes and Program Specific Outcomes. A predefined correlation scale is used to indicate the strength of relationship between them. This mapping helps in measuring how individual subjects contribute to overall program objectives.

- ***Phase 3: Collection of Assessment Data***

  Student performance data is collected through various evaluation methods such as internal tests, assignments, practical examinations, semester examinations, and feedback forms. Faculty members enter this information into the system. Data validation mechanisms ensure that incorrect or incomplete entries are minimized.

- ***Phase 4: Computation of Course Outcome Attainment***

  The system calculates the attainment level of each Course Outcome automatically. The calculation is based on student performance and predefined target criteria. The backend processing logic ensures that the calculation is consistent and free from manual errors. This automated process saves time and improves accuracy.

- ***Phase 5: Computation of Program-Level Attainment***

  After determining Course Outcome attainment, the system calculates Program Outcome and Program Specific Outcome attainment. This is done by applying mapping weight values and computing average contribution levels. These results provide an overall picture of academic achievement at the program level.

- ***Phase 6: Performance Gap Identification***

  The calculated attainment values are compared with expected target benchmarks. If the results fall below the expected level, the system highlights the performance gap. This helps academic authorities take corrective measures and implement improvement strategies for future cycles.

6.3. Student Stress Evaluation Process

In addition to academic measurement, the system includes a stress evaluation feature. A structured questionnaire is designed to collect student responses related to academic pressure and workload.

The process includes:

- Preparing relevant survey questions.
- Collecting responses securely.
- Analyzing patterns and trends.
- Generating summary reports.

This feature helps the institution understand the relationship between stress levels and academic performance.

6.4. Role-Based Operational Flow

The system operates through a controlled role-based structure to maintain accountability and security. The workflow includes:

- Faculty members entering academic data.
- Head of Department reviewing and verifying entries.
- Coordinator reviewing processes under NBA Criterion 3.
- Auditor reviewing final documents and giving remarks.

This structured flow ensures proper validation, secure access, and organized documentation.

6.5. System Verification and Testing

To ensure smooth functioning, multiple testing techniques were applied during development:

- Module-level testing to verify individual functions.
- Integration testing to ensure compatibility between modules.
- Complete system testing for overall performance.
- User-level testing for practical validation.

All detected issues were corrected before final implementation.

CHAPTER 7. MODULES

The OBE Tracking System is designed using a modular architecture approach. Each module is responsible for handling specific functionalities of the system. The modular structure improves maintainability, scalability, and clarity of the overall system.

The system consists of backend modules responsible for data processing and logic implementation, and frontend modules responsible for user interaction and visualization. All modules work together in an integrated manner to provide a complete and automated OBE management solution.

7.1. Backend Modules

7.1.1. Users Module:

The Users module is responsible for managing user authentication and authorization within the system. It handles different types of users such as:

- Administrator.
- Head of Department (HOD).
- Coordinator
- Faculty.
- Auditor
- Student.

This module manages:

- User registration.
- Secure login and logout.
- Role-based access control.
- Profile management.

Each user is assigned specific permissions according to their role. For example, only faculty members can enter marks, while only administrators can configure academic structures. This ensures data security and controlled access.

7.1.2. Academics Module:

The Academics module manages all core academic data of the institution. It acts as the foundation of the entire system. This module includes:

- Batch and Academic Year management.
- Semester configuration.
- Department creation.
- Course allocation.
- Scheme and curriculum mapping.

It ensures that all academic structures are properly defined before assessments and attainment calculations are performed.

This module helps in maintaining uniformity and consistency in academic data.

7.1.3. CIS Master Module:

The Course Index Sheet (CIS) module allows faculty to manage CIS master data such as:

- Define Course Outcomes (COs).
- Map COs to Program Outcomes (POs).
- Map COs to Program Specific Outcomes (PSOs).
- Assign weightages.

This mapping is stored systematically and used later for attainment calculation. The standardized digital format ensures uniform mapping across all courses.

7.1.4. Assessments Module:

The Assessments module handles all evaluation-related activities. It includes:

- Creation of internal assessments.
- External examination data entry.
- Question-wise CO mapping.
- Student marks entry.

Faculty members can securely enter marks through structured forms. The system validates the data and stores it in a centralized database.

This module reduces manual calculation errors and ensures accurate linking between questions and course outcomes.

7.1.5. Attainment Module:

The Attainment module is responsible for calculating direct and overall attainment levels. It performs:

- CO attainment calculation.
- PO attainment calculation.
- Gap analysis.

The system automatically applies predefined formulas for calculating direct attainment:

Direct CO = (0.4 × Internal CO) + (0.6 × External CO)

It then derives PO attainment using CO–PO mapping relationships.

This module eliminates dependency on Excel formulas and ensures consistent results across all departments.

7.1.6. Indirect Attainment Module:

Indirect attainment is calculated using survey feedback data such as:

- Course Exit Survey (Course level).
- Curricular Activity Feedback (Expert Lecture, Industry Visit, etc).
- Activity Resource Person Feedback.
- Alumni Feedback.
- Program Exit Survey.

The module converts qualitative feedback into quantitative scores and integrates it with direct attainment values to calculate overall attainment. This ensures a balanced evaluation approach.

7.1.7. Surveys Module:

The Surveys module manages survey creation and response collection. It allows administrators and faculty to:

- Manage dynamic questionnaires.
- Define survey types.
- Collect student responses.

The system stores survey responses securely and generates statistical summaries automatically.

7.1.8. Stress Module:

The Stress module is a unique feature of this system. It focuses on monitoring student stress levels through structured questionnaires. It performs:

- Stress survey management.
- Response evaluation.
- Stress level categorization.
- Report generation.

This module helps in identifying students who may require academic or emotional support.

7.1.9. Reports Module:

The Reports module is responsible for generating structured academic reports. It generates:

- CIS Reports (Direct Attainment).
- Indirect Attainment Reports.
- PO / PSO Attainment Reports.
- Stress Analysis Reports.

Reports can be exported in:

- Excel format.

This module plays a critical role during accreditation audits.

7.1.10. Teaching Plan Module:

The Teaching Plan module allows faculty members to:

- Track syllabus coverage.
- Update completion status.

This ensures that teaching activities are properly planned and monitored.

7.1.11. Audit Module:

The Audit module maintains system transparency and accountability. It tracks:

- User login history.
- Data modifications.
- Record updates.
- System activities.

This improves security and ensures traceability of changes.

7.1.12. Bulk Upload Module:

The Bulk Upload module reduces manual effort by allowing large datasets to be uploaded at once. It supports:

- Student list upload.
- Marks upload.

Excel templates are provided to ensure structured data import.

7.1.13. Dashboard Module:

The system provides graphical dashboards for:

- CO attainment trends.
- PO / PSO attainment trends.
- Survey status.

This helps management in quick decision-making.

7.1.14. Core Module:

The Core module contains overall system configuration settings such as:

- Global system settings.
- URL routing.
- Server configurations.
- Application setup.

It acts as the backbone of the system.

7.2. Frontend Modules (React Web Application)

7.2.1. Admin User:

The Admin user provides management-level control. It allows administrators to:

- Manage users.
- Monitor system activities.

7.2.2. HOD User:

The HOD user is designed specifically for department heads. It provides:

- Department-wise attainment monitoring.
- Target setting.
- Faculty performance overview.
- Consolidated reports.

7.2.3. Faculty User:

The Faculty user enables teachers to:

- Manage assigned courses.
- Enter assessment marks.
- Define CO–PO mapping.
- View attainment results.
- Generate course-level reports.

7.2.4. Auditor User:

The Auditor user allows auditors to

- View reports.
- View Mapping.
- Give remarks.

7.2.5. Student User:

The Student user allows students to:

- Participate in surveys.
- View stress-related guidance.

7.2.6. Common Module:

The Common module includes shared components such as:

- Login page.
- Unauthorized access page.
- Navigation layout.
- API service integration.

This module ensures uniform design and communication between frontend and backend.

7.3. Inter-Module Interaction

All modules interact in a structured workflow:

- Academics module defines courses and semesters.
- CIS module defines CO–PO mappings.
- Assessments module records student marks.
- Attainment module calculates results.
- Survey module provides indirect data.
- Reports module generates final documentation.
- Dashboard displays summarized data.

This integrated flow ensures smooth operation of the entire system.

CHAPTER 8. SOFTWARE REQUIREMENT SPECIFICATION

8.1. System Requirements

The OBE Tracking System is developed as a web-based application to automate the process of Outcome Based Education implementation. The system uses modern frontend and backend technologies to ensure secure data handling, accurate attainment calculation, and smooth user experience.

To run the system efficiently, certain hardware and software requirements must be fulfilled.

8.1.1. Minimum Specification Requirement:

The minimum specifications are the basic requirements needed to run the system during development.

- ***Hardware Requirements:***
    - **Processor:** Dual Core Processor.
    - **RAM:** 4 GB.
    - **Hard Disk:** Minimum 20 GB free space.
    - **Internet Connection:** Stable broadband connection.

- ***Software Requirements:***
    - **Operating System:** Windows 10 / Linux.
    - **Web Browser:** Google Chrome / Microsoft Edge / Mozilla Firefox.
    - **Frontend Technology:** React.
    - **UI Framework:** Bootstrap.
    - **Backend Framework:** Django REST Framework.
    - **Development Database:** SQLite.
    - **Authentication Method:** JSON Web Token.
    - **API Testing Tool:** ThunderClient extension, VS Code.

These specifications are sufficient for developing and testing the system within an institutional environment.

8.1.2. Recommended Specification Requirement:

For better performance and handling large amounts of student data, the following specifications are recommended.

- ***Hardware Requirements:***
    - **Processor:** Intel i5 or higher.
    - **RAM:** 8 GB or more.
    - **Hard Disk:** Minimum 50 GB free space.
    - **Internet Connection:** High-speed Internet connection.

- ***Software Requirements:***
    - Updated Operating System.
    - Latest version of Web Browser.
    - Backend Server for deployment.
    - **Deployment Database:** PostgreSQL.

The recommended configuration ensures faster API response, smooth multi-user access, and better database performance.

8.2. Data Requirements

The system stores and processes different types of academic and administrative data. Proper data management is necessary to maintain accuracy, security, and reliability.

The major data stored in the system includes:

- User details (Admin, HOD, Faculty, Coordinator, Auditor and Student).
- Department and semester information.
- Course details.
- Course Outcomes (COs).
- Program Outcomes (POs).
- Program Specific Outcomes (PSOs).
- CO–PO and CO–PSO mapping values.
- Internal and external assessment marks.
- Attainment calculation results.
- Survey responses.
- Stress analysis data.

During development phase, data is stored in SQLite database, and PostgreSQL database is used for better scalability and reliability. All data access is controlled using role-based authentication with JWT and a custom user model in the backend.

8.3. Functional Requirements

Functional requirements describe what the system must do. The OBE Tracking System must be able to:

- Provide secure login and logout functionality.
- Implement role-based access control.
- Allow admin to manage academic structure.
- Allow faculty to define COs, POs, and PSOs.
- Perform CO–PO and CO–PSO mapping.
- Enter and store internal and external assessment marks.
- Automatically calculate CO attainment.
- Automatically calculate PO and PSO attainment.
- Compare attainment values with predefined targets.
- Generate gap analysis reports.
- Conduct surveys and collect feedback.
- Perform indirect attainment calculation.
- Analyze stress survey responses.
- Generate reports in Excel format.
- Maintain audit logs for system activities.

These functions ensure full automation of the OBE implementation process.

8.4. External Requirements

External requirements define how the system interacts with users and other environments.

8.4.1. User Interface Requirements:

- Simple and user-friendly interface.
- Responsive design using Bootstrap.
- Clear navigation menus.
- Separate dashboards for Admin, HOD, Faculty, Coordinator and Auditor.
- Proper form validation.

8.4.2. Backend Interface Requirements:

- REST APIs developed using Django REST Framework.
- JSON-based communication between frontend and backend.

- Secure token-based authentication using JWT.

8.4.3. Security Requirements:

- Encrypted password storage.
- JWT-based authentication.
- Role-based access restrictions.
- Secure database transactions.
- Protection against unauthorized access.

8.4.4. Performance Requirements:

- Fast data processing.
- Quick report generation.
- Support for multiple users simultaneously.
- Stable API response time.

8.4.5. Operational Requirements:

- The system works through a web browser.
- No installation required on client machines.
- Accessible within institutional network.
- Easy to maintain and update.

CHAPTER 9. SYSTEM MODELLING

9.1. System Design Diagram

System Design Diagram represents the overall structure and working of the system in a graphical form. It shows how different components of the system are connected and how data flows between them. It helps to understand the system architecture and interaction between users and system modules. In the OBE Tracking System, it explains how data is processed, stored, and managed within the system.

- **Level 0: Data Flow Diagram**

  Data flow diagram (DFD) shows logical flow of system. Data flow diagrams are more disciplined and structured. Data flow diagrams are quite readable independent of complexity of system. The DFDs of the project as shown below:

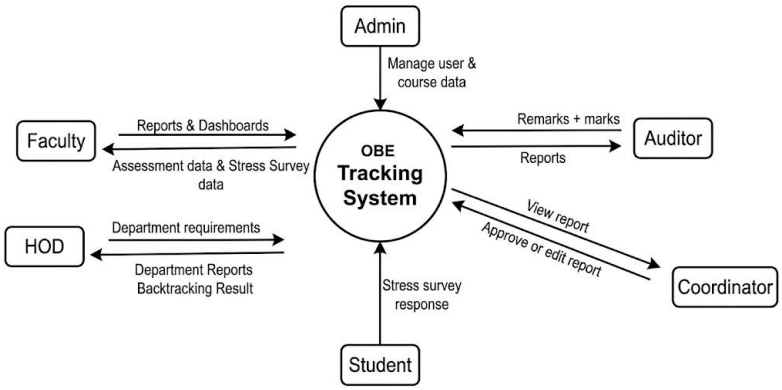


Fig. No. 2. DFD Level - 0 of OBE Tracking System

- **Level 1: Data Flow Diagram**

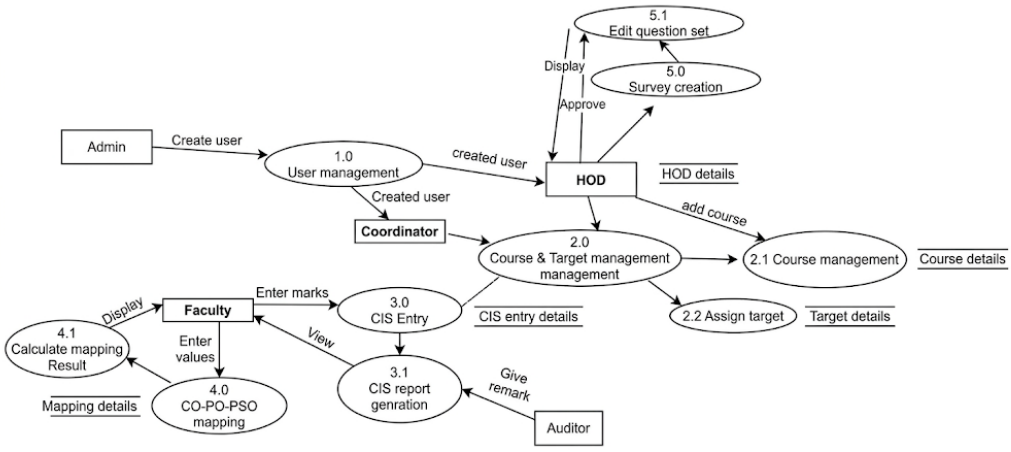


Fig. No. 3. DFD Level - 1 of OBE Tracking System

9.2. Use Case Diagram

A use case diagram in the type of behavioral diagram. It is defined by as well as created from a use-case analysis. The main goal is to constitute the graphical structure of the overall functionality provided by a system in terms of actors and dependencies between the use cases. Use case diagram of OBE Tracking System shown in Figure 9.2.

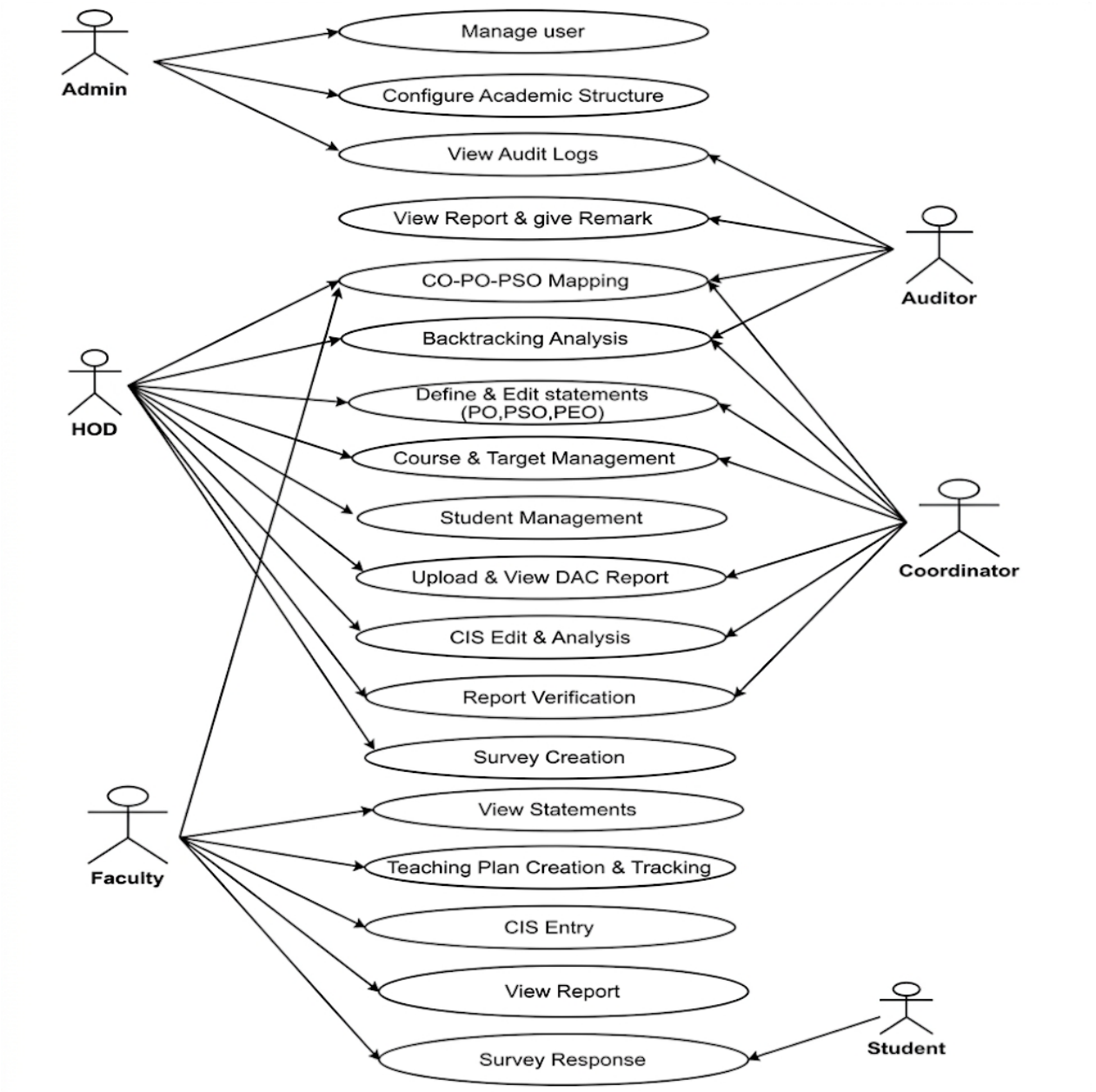


Fig. No. 4. Use Case diagram of OBE Tracking System

9.3. System Architecture

The System Architecture Diagram describes the overall structure of the system and how different components are connected. It shows the interaction between the user interface, application server, and database. This diagram helps to understand how the system processes requests and manages data.

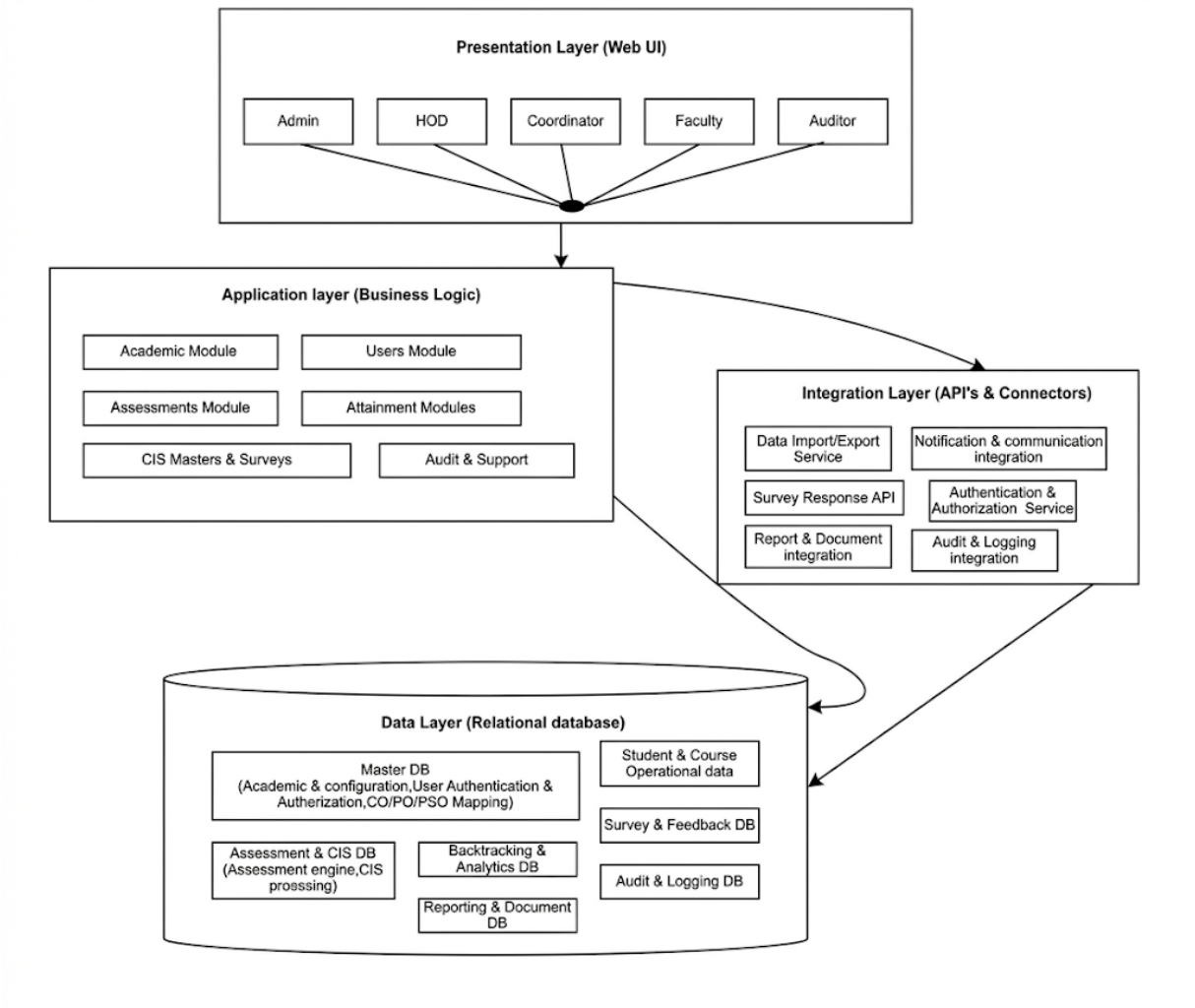


Fig. No. 5. System Architecture of OBE Tracking System

9.4. UML DIAGRAM

Unified Modelling Language (UML) is a standardized modelling language used to design a software-based system model. UML includes a set of graphical diagrams.

9.4.1. Class Diagram:

A Class Diagram is a type of UML diagram that represents the structure of the system by showing different classes and the relationships between them. It describes the attributes and

functions of each class in the system.

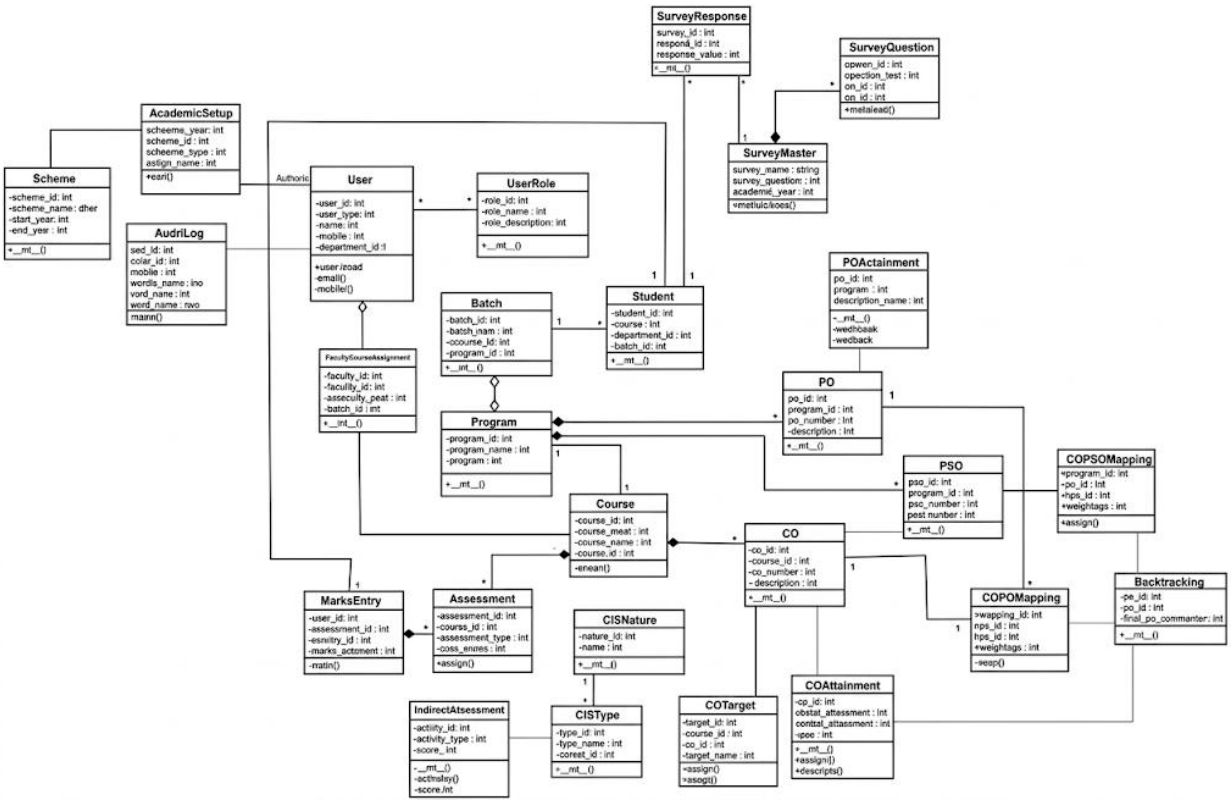


Fig. No. 6. Class Diagram of OBE Tracking System

9.4.2. Sequence Diagram 1:

A Sequence Diagram shows the interaction between different objects in the system in a sequential order. It represents how messages are passed between components to perform a particular function. The sequence diagram helps in understanding the flow of operations in the system step by step.

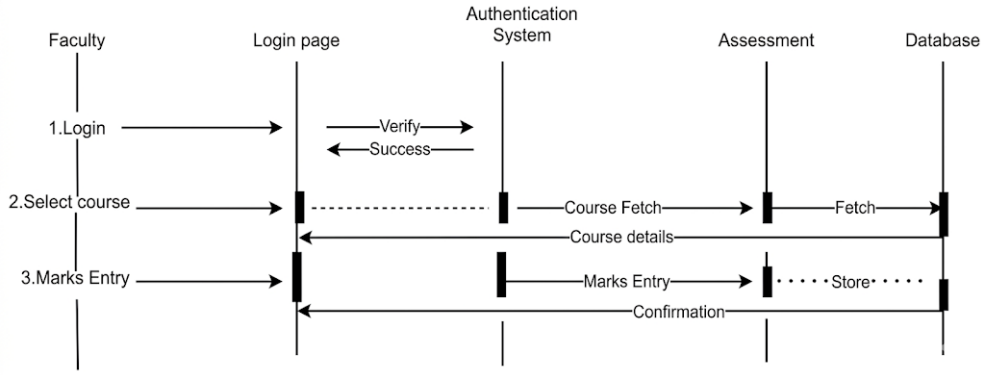


Fig. No. 7. Sequence Diagram 1 of OBE Tracking System

9.4.3. Sequence Diagram 2:

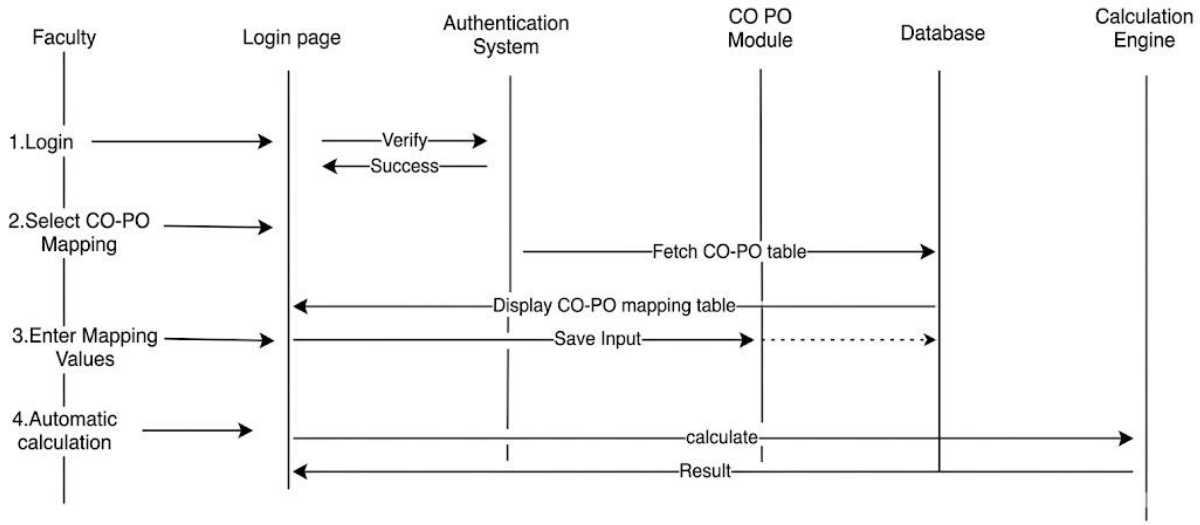


Fig. No. 8. Sequence Diagram 2 of OBE Tracking System

9.4.4. Sequence Diagram 3:

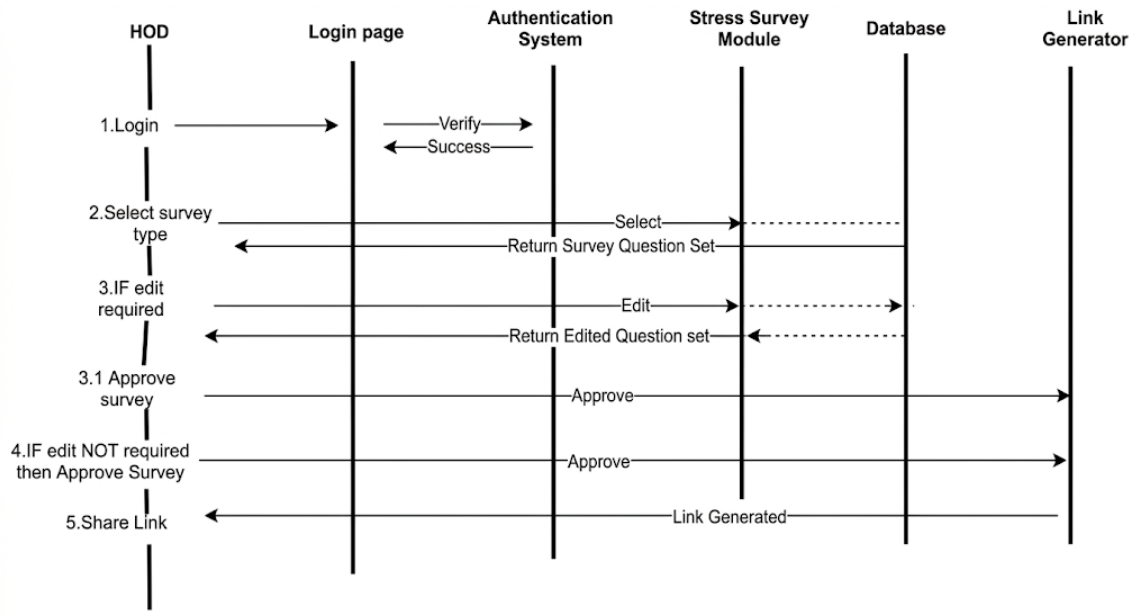


Fig. No. 9. Sequence Diagram 2 of OBE Tracking System

CHAPTER 10. IMPLEMENTATION DETAIL

10.1. Introduction

This chapter explains how the OBE Tracking System was practically developed and implemented. The system is designed as a web-based application to automate the process of Outcome Based Education implementation under NBA Criterion 3 in academic institutions.

The implementation focuses on providing a secure, user-friendly, and automated system that reduces manual work and improves accuracy in attainment calculations. The system follows a three-layer architecture consisting of frontend, backend, and database layers.

10.2. Frontend Implementation

The frontend of the system is developed using **React**. React is used to create a dynamic and interactive user interface. It allows the development of reusable components such as login pages, dashboards, data entry interfaces, mapping interfaces, and report pages.

For styling and layout design, **Bootstrap** is used. Bootstrap helps in creating responsive web pages that adjust properly on different screen sizes. It is used to design:

- Navigation bars.
- Forms.
- Tables.
- Buttons.
- Dashboard layouts.

The frontend communicates with the backend using REST APIs. All requests are sent through HTTP methods and responses are received in JSON format. The user interface is designed to be simple and easy to understand so that users can operate the system without difficulty.

10.3. Backend Implementation

The backend of the system is developed using Django REST Framework. It is responsible for handling all business logic, data validation, authentication, and attainment calculations. The backend is structured using:

- Models to define database structure.
- Serializers to convert data into JSON format.
- Views to implement business logic.

- URL routing to manage API endpoints.

All calculations related to CO attainment, PO attainment, and indirect attainment are implemented in backend logic. This ensures accurate and consistent results across all courses and departments. The backend also manages role-based access control and ensures that users can only access features based on their assigned roles.

10.4. Database Implementation

10.4.1. Development Phase Database:

During the development phase, the system uses SQLite as the database. SQLite is lightweight and easy to configure. It does not require a separate server and works smoothly with Django. It is suitable for testing and development purposes.

10.4.2. Deployment Phase Database:

During the deployment phase, the system uses PostgreSQL. PostgreSQL is selected for deployment because it provides:

- Better performance.
- Support for large datasets.
- High security.
- Multi-user access capability.

This ensures that the system works efficiently in real institutional environments.

10.5. Authentication and Security Implementation

The system uses JSON Web Token (JWT) for secure authentication. A custom user model is implemented in Django to manage different user roles such as:

- Admin.
- Head of Department (HOD).
- Faculty.
- Coordinator.

Authentication Process:

- The user enters login credentials.
- The backend verifies the username and password.

- If valid, a JWT token is generated.
- The token is sent to the frontend.
- The token is included in all protected API requests.
- The backend verifies the token before allowing access.

This mechanism ensures secure login and prevents unauthorized access to the system.

10.6. Module Implementation

Each module of the system was implemented separately and later integrated.

- **User Management Module:** Handles user creation, role assignment, login, and logout.
- **Academic Configuration Module:** Manages academic years, semesters, departments, and course allocation.
- **CIS Module:** Allows faculty to define Course Outcomes and perform CO–PO–PSO mapping.
- **Assessment Module:** Handles internal and external marks entry with validation.
- **Attainment Module:** Automatically calculates direct and indirect attainment levels.
- **Survey and Stress Module:** Collects feedback and evaluates student stress levels.
- **Report Module:** Generates CO attainment reports, PO attainment reports, and gap analysis reports in structured format.

10.7. API Integration

The frontend and backend communicate using REST APIs. When a user performs any action, such as entering marks or viewing reports:

- React sends a request to the backend API.
- The backend processes the request.
- Data is stored or retrieved from the database.
- A JSON response is returned to the frontend.
- The frontend displays the result to the user.

This integration ensures smooth and fast data transfer between layers.

10.8. System Workflow

The overall implementation workflow is as follows:

- User logs into the system.
- Authentication is verified using JWT.
- Faculty enters academic data and assessment marks.
- Data is stored in the database.
- Backend calculates CO attainment.
- PO and PSO attainment are derived from mapping.
- Reports are generated.
- HOD reviews and approves reports.

This structured workflow ensures proper validation and accountability.

10.9. Advantages of Implementation

- Secure token-based authentication.
- Custom role management.
- Automated attainment calculation.
- Centralized database storage.
- Reduced manual errors.
- Easy report generation.
- Scalable deployment using PostgreSQL.
- User-friendly interface.

CHAPTER 11. COST ESTIMATION

11.1. Introduction

Cost estimation is an essential part of software project planning. It helps in predicting the effort, development time, and overall cost required to build the system.

For the OBE Tracking System with Stress Analysis, the estimation is performed using:

- Function Point Analysis (FPA).
- COCOMO Model (Organic Mode).

This approach provides a structured and standardized way to estimate software size and development effort.

11.2. Function Point Analysis (FPA)

Function Point Analysis is used to measure the functional size of the system based on user interactions and data processing.

11.2.1. Unadjusted Function Points (UFP):

The system functionalities are categorized and grouped logically to avoid overestimation.

Function Category	Description	Count	Weight	UFP
Internal Logical Files (ILF)	Users, Academics, OBE, Targets, Assessment, Progress, Attainment, Surveys, Reporting, Stress, System	11	10	110
External Interface Files (EIF)	Deployment platform, PostgreSQL, Excel integrations	2	7	14
User Inputs (EI)	System setup, mappings, targets, Excel uploads, survey submissions	10	4	40
User Outputs (EO)	Dashboards, reports, audit logs	5	5	25
User Inquiries (EQ)	Search, filters, status checks, previews	4	4	16

Total UFP = 205

Table No. 1. Unadjusted Function Points

11.3. Technical Complexity Factor (TCF)

The system is evaluated based on various technical factors.

Degree of Influence (DI) = 49

Formula:

TCF = 0.65 + (0.01 × DI)

TCF = 0.65 + (0.01 × 49) = 1.14

11.4. Adjusted Function Points (FP)

FP = UFP × TCF

FP = 205 × 1.14 = 233.7

11.5. Conversion to KLOC

Using standard conversion for Python/Django:

KLOC = (FP × 55) / 1000

KLOC = (233.7 × 55) / 1000 = 12.85 KLOC

11.6. Effort Estimation using COCOMO

Organic Mode Constants:

a1 = 2.4, a2 = 1.05

b1 = 2.5, b2 = 0.38

11.6.1. Effort Calculation

Effort = 2.4 × (12.85) ^ 1.05

Effort = 34.85 Person-Months

11.6.2. Development Time

Time = 2.5 × (34.85) ^ 0.38

Time = 9.6 Months

11.7. Development Cost Estimation

Assuming average developer cost = Rs.10,000 per month

Total Cost = 9.6 × 10,000

Total Cost = Rs. 96,000

11.8. Real Project Comparison

Metric	Estimated	Actual
Team Size	~3.6	4 Developers
Duration	9.6 Months	114 Days (~3.8 Months)
Effort	34.85 PM	15.2 PM

Actual Effort = 4 × 3.8 = 15.2 Person - Months

Table No. 2. Real Project Comparison

The variation between estimated and actual effort is due to:

- Use of modern frameworks (Django and React).
- Reuse of existing libraries (Excel handling, authentication).
- Efficient coordination within a small development team.
- Reduced overhead compared to traditional development models.

The variation is also influenced by conservative estimation in Function Point Analysis, which may slightly overestimate system size compared to actual implementation.

11.9. Deployment Cost Estimation

The system can be deployed on cloud infrastructure using Amazon Web Services.

Estimated monthly infrastructure cost:

- Amazon EC2: Rs.600 to Rs.850
- Amazon RDS: Rs.1000 to Rs.1700
- Storage and network: Rs.150 to Rs.500

Total monthly deployment cost: Rs.1750 to Rs.5000

11.10. Domain Name Cost

To make the system accessible via a public URL, a domain name is required.

Typical domain cost from providers like GoDaddy:

- **First year:** Rs.100 to Rs.1000 (depending on offers and domain type)
- **Renewal:** Rs.1000 to Rs.1500 per year.

11.11. Overall Cost Estimation

Component	Cost
Development Cost	**Rs. 96,000**
Deployment Cost (Annual)	**Rs. 21,000 to Rs. 60,000**
Domain Cost (Annual)	**Rs. 1000 to Rs.1500**

Table No. 3. Overall Cost Estimation

Total Estimated Cost (First Year)

Minimum Cost:

Rs. 96,000 + Rs. 21,000 + Rs. 1000 = Rs. 1,18,000

Maximum Cost:

Rs. 96,000 + Rs. 60,000 + Rs. 1500 = Rs. 1,57,500

11.12. Conclusion

The cost estimation shows that the OBE Tracking System is a medium-scale web application with moderate development effort and cost. While theoretical models estimate a higher development duration, the actual project was completed efficiently due to modern technologies and optimized development practices.

The inclusion of deployment and domain costs provides a more realistic view of total system expenditure in a production environment.

CHAPTER 12. SOURCE CODE DESCRIPTION

12.1. Introduction

Source code represents the actual implementation of the system functionalities. It defines how different modules of the Outcome Based Education (OBE) Tracking System are structured, configured, and integrated.

The system is divided into two main parts:

- Backend (Django REST Framework).
- Frontend (React).

The source code is organized in a modular manner to ensure scalability, maintainability, and ease of development.

12.1.1. Project Directory Structure:

The overall structure of the system is as follows:

OBE-TRACKING-SYSTEM/

```
│
│
├── backend/
│   ├── academics/
│   ├── assessments/
│   ├── attainment/
│   ├── audit/
│   ├── bulk_upload/
│   ├── cis_master/
│   ├── core/
│   │   ├── __init__.py
```

```
│   │   ├── __pycache__/
│   │   ├── asgi.py
│   │   ├── settings.py
│   │   ├── urls.py
│   │   └── wsgi.py
│   ├── indirect_attainment/
│   ├── media/
│   ├── notifications/
│   ├── reports/
│   ├── stress/
│   ├── surveys/
│   ├── teaching_plan/
│   ├── users/
│   ├── manage.py
│   └── requirements.txt
│
├── frontend/
│   └── webapp/
│       ├── build/
│       ├── node_modules/
│       ├── public/
│       ├── package.json
│       │
│       └── src/
```

```
│       ├── App.js
│       ├── App.css
│       ├── App.test.js
│       ├── index.js
│       ├── index.css
│       ├── api.js
│       │
│       ├── assets/
│       │   └── images/
│       │       ├── <images used in the system>
│       │       └── index.js
│       │
│       ├── components/
│       │   ├── DashboardRedirect.js
│       │   ├── GlobalAlert.js
│       │   ├── filters/
│       │   ├── header/
│       │   ├── layout/
│       │   └── sidebar/
│       │
│       ├── context/
│       │   └── FilterContext.js
│       │
│       ├── pages/
```

```
│   │   ├── admin/
│   │   ├── auditor/
│   │   ├── common/
│   │   ├── coordinator/
│   │   ├── faculty/
│   │   ├── hod/
│   │   │   ├── attainment/
│   │   │   ├── backtracking/
│   │   │   ├── course/
│   │   │   ├── dac/
│   │   │   ├── dashboard/
│   │   │   ├── feedback/
│   │   │   ├── mapping/
│   │   │   ├── plan/
│   │   │   ├── report/
│   │   │   ├── stress/
│   │   │   ├── student/
│   │   │   ├── survey/
│   │   │   └── target/
│   │   │
│   │   └── student/
│   │
│   ├── services/
│   │   ├── authService.js
```

```
│   │   ├── feedbackService.js
│   │   └── stressService.js
│   │
│   └── utils/
│       ├── auth.js
│       ├── axios.js
│       └── semesterUtils.js
```

The backend is divided into multiple Django applications, each handling a specific functionality.

The frontend follows a component-based structure using React.

12.2. Backend Source Code Structure

The backend is developed using Django REST Framework and is organized into modular applications such as academics, assessments, attainment, audit, and others.

Each module follows a standard structure containing:

- models.py (database schema).
- views.py (business logic).
- serializers.py (data conversion).
- urls.py (API routing).

12.2.1. Core Configuration (settings.py):

The settings.py file contains global configuration for the Django project.

```
INSTALLED_APPS = [
    'rest_framework',
    'corsheaders',
    'users',
```

```
    'academics',
    'cis_master',
    'assessments',
    'attainment',
    'reports',
    'stress',
    'surveys',
    'teaching_plan',
    'audit',
    'bulk_upload',
    'notifications',
]
```

This configuration registers all modules used in the system.

Database configuration is handled dynamically:

```
DATABASES = {
    'default': dj_database_url.config(
        default=f'sqlite:///{BASE_DIR / "db.sqlite3"}',
        conn_max_age=600
    )
}
```

This allows switching between SQLite (development) and PostgreSQL (deployment).

JWT authentication is configured as:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
```

```
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

Static and media file handling is also configured for deployment using WhiteNoise.

12.2.2. URL Routing (core/urls.py):

The urls.py file acts as a central routing system that connects all modules.

```
urlpatterns = [
    path('api/users/', include('users.urls')),
    path('api/academics/', include('academics.urls')),
    path('api/assessments/', include('assessments.urls')),
    path('api/attainment/', include('attainment.urls')),
    path('api/reports/', include('reports.urls')),
    path('api/stress/', include('stress.urls')),
    path('api/surveys/', include('surveys.urls')),
    path('api/teaching-plan/', include('teaching_plan.urls')),
    path('api/audit/', include('audit.urls')),
    path('api/bulk_upload/', include('bulk_upload.urls')),
    path('api/notifications/', include('notifications.urls')),
]
```

Each module is exposed through a dedicated API endpoint, ensuring modular and scalable routing.

12.3. Frontend Source Code Structure

The frontend is developed using React and is organized into:

- Pages (screens).

- Components (reusable ui elements).
- Services (API communication).
- Utils (helper functions).

12.3.1. Main Application File (App.js):

The App.js file defines routing for the entire application.

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Login />} />
    <Route path="/dashboard" element={<DashboardRedirect />} />
    <Route path="/admin-dashboard" element={<Layout><AdminDashboard /></Layout>} />
  </Routes>
</BrowserRouter>
```

Key features:

- Role-based routing (Admin, Faculty, HOD, Auditor, Student).
- Lazy loading for performance optimization.
- Centralized navigation control.

12.3.2. Service Layer (authService.js):

The service layer handles communication with backend APIs.

```
export const login = async (email, password) => {
    const response = await api.post("/users/login/", { email, password });

    localStorage.setItem("access", response.data.access);
    localStorage.setItem("refresh", response.data.refresh);
```

```
    localStorage.setItem("user", JSON.stringify(response.data.user));


    return response.data;
};
```

This abstracts API calls and manages authentication tokens.

12.3.3. Utility Functions (auth.js):

Utility functions handle common operations such as user management.

```
export const getLoggedInUser = () => {
    const user = localStorage.getItem("user") || sessionStorage.getItem("user");
    return user ? JSON.parse(user) : null;
};
```

These functions simplify state management across the application.

12.3.4. Axios Configuration (axios.js):

Axios is configured to handle API requests globally.

```
const api = axios.create({
    baseURL: "http://127.0.0.1:8000/api/",
});
```

JWT token is automatically attached to requests:

```
config.headers.Authorization = `Bearer ${token}`;
```

Unauthorized access is handled centrally:

```
if (error.response && error.response.status === 401) {
    localStorage.clear();
    window.location.href = "/";
```

}

12.4. Database Implementation

The system uses:

- SQLite during development.
- PostgreSQL during deployment.

Django ORM is used for database operations, enabling:

- Data insertion.
- Updates.
- Retrieval.

Without writing raw SQL queries.

12.5. Authentication Implementation

JWT-based authentication is implemented:

- User logs in → token generated.
- Token stored in browser storage.
- Token attached to every API request.

Only authorized users can access protected resources.

12.6. Dependencies

12.6.1. Backend Dependencies (requirements.txt):

Django==6.0

djangorestframework==3.16.1

djangorestframework_simplejwt==5.5.1

psycopg2-binary==2.9.10

pandas==2.2.3

These libraries support API development, authentication, database connectivity, and data processing.

12.6.2. Frontend Dependencies (package.json)

react

axios

bootstrap

react-router-dom

react-icons

These libraries support UI development, routing, API communication, and styling.

CHAPTER 13. TESTING

13.1. Introduction

Testing is an essential phase of the Software Development Life Cycle (SDLC). It ensures that the developed system works correctly according to the specified requirements and performs all operations accurately. The purpose of testing is to detect errors, verify functionality, and confirm that the software meets the needs of the users.

In this project, a systematic testing process was carried out to ensure the reliability and correctness of the application. Different modules such as Role Based Access Control (RBAC), Login Authentication, Stress Survey Module, Attainment Module, Master Data Module, and Report Generation Module were tested.

The testing process included designing test cases, executing them on different modules, and comparing the actual results with the expected results. The testing activities were performed during various stages of development as described in the project plan. This helped in identifying issues early and improving the quality of the system.

13.2. Objectives of Testing

The main objectives of software testing in this project are:

- To verify that the system functions according to the specified requirements.
- To detect errors and bugs in different modules of the system.
- To ensure that authentication and role-based access control work correctly.
- To validate the stress survey responses and calculations.
- To check the integration between frontend and backend components.
- To ensure accurate report generation and data handling.
- To improve the reliability and performance of the application.

13.3. Testing Strategy

A structured testing strategy was followed in this project to ensure proper validation of the system. The strategy included the following testing levels:

13.3.1. Unit Testing:

Unit testing involves testing individual components or modules of the system separately. In this project, unit testing was conducted for several modules such as authentication, survey creation, stress calculation, and attainment logic.

The purpose of unit testing was to ensure that each module performs its intended functionality without errors. The following modules were tested individually:

- RBAC (Role Based Access Control).
- Login Authentication Module.
- Stress Survey Module.
- Stress Calculation Module.
- Master Data Module.
- Attainment Module.
- Report Generation Module.

Unit testing helped detect issues early in development and ensured that each component worked correctly before integration.

13.3.2. Integration Testing:

Integration testing was performed after the completion of unit testing. In this stage, different modules were combined and tested together to verify proper interaction between them. The integration process included:

- Connecting React frontend with Django REST APIs.
- Integrating Login API with the user interface.
- Connecting Stress survey responses with stress calculation logic.
- Integrating Master Data module with Attainment calculations.
- Linking RBAC authentication with user roles and permissions.

This testing ensured that data flows correctly between modules and that the system behaves as expected when modules interact.

13.3.3. System Testing:

System testing was performed after integrating all modules of the system. In this stage, the entire application was tested as a complete system to verify that all functionalities work correctly. The following operations were verified during system testing:

- User authentication and login.
- Role-based access control for Admin, Faculty, and HOD.
- Stress survey creation and submission.
- Stress calculation and analysis.
- Attainment calculation based on academic data.

- Report generation and data visualization.

System testing ensured that the complete application met the functional requirements and worked efficiently in real scenarios.

13.3.4. User Acceptance Testing:

User Acceptance Testing (UAT) was carried out to validate that the application meets the functional and operational requirements from the end-users' perspective. Key stakeholders, including faculty, administrators, and HODs, participated in testing. The main focus was on usability, workflow efficiency, and overall system functionality. Activities performed during UAT included:

- Verifying the login process and role-based access for different users (Admin, Faculty, HOD).
- Checking the creation, submission, and accuracy of stress surveys.
- Validating the correctness of stress calculation and attainment results.
- Ensuring proper generation, format, and accuracy of reports and visualizations.
- Confirming that the system responds correctly to typical user operations and handles errors gracefully.
- Testing the integration of different modules from an end-user perspective.
- Collecting feedback on user interface, ease of navigation, and overall user experience.
- Identifying any gaps between system functionality and user expectations for final refinements.

UAT ensured that the system is user-friendly, meets organizational requirements, and is ready for deployment in a real operational environment.

Each module of the system was tested individually first, and later all modules were integrated and tested together.

13.4. Test Case Design

Test cases were designed for different modules based on the project requirements and functionality. Each test case includes information such as test case number, module name, input conditions, expected output, actual output, and test status.

These test cases helped verify whether the system produced correct outputs under different

conditions.

13.5. Login Module Test Cases

Sr. No.	Test Case Description	Input Data	Expected Results	Actual Results	Status
1	Login with valid credentials	Correct email and password	User should login successfully	Login successful	**Pass**
2	Login with incorrect password	Correct email and wrong password	Error message displayed	Error message shown	**Pass**
3	Login with invalid email	Invalid email address	Invalid credentials message	Invalid credentials message shown	**Pass**
4	Login with empty fields	No input provided	Validation message displayed	Validation message shown	**Pass**

Table No. 4. Login Module Test Cases

13.6. RBAC Module Test Cases

Sr. No.	Test Case Description	Input Data	Expected Results	Actual Results	Status
1	Admin login access	Admin Credentials	Access to admin dashboard	Access to admin dashboard successful	**Pass**
2	Faculty login access	Faculty Credentials	Access to faculty dashboard	Access to faculty dashboard successful	**Pass**

3	Unauthorized access	Invalid role	Access denied message	Access denied message shown	**Pass**

Table No. 5. RBAC Module Test Cases

13.7. Stress Survey Module Test Cases

Sr. No.	Test Case Description	Input Data	Expected Results	Actual Results	Status
1	Create stress survey	Survey details	Survey created successfully	Survey created successfully	**Pass**
2	Submit survey responses	Answers to questions	Responses stored in database	Responses stored in database	**Pass**
3	Submit incomplete survey	Missing responses	Validation error message	Validation error message shown	**Pass**
4	Multiple survey responses	Large dataset	System processes data correctly	System processes data correctly	**Pass**

Table No. 6. Stress Survey Module Test Cases

13.8. Stress Calculation Module Test Cases

Sr. No.	Test Case Description	Input Data	Expected Results	Actual Results	Status
1	Calculate stress score	Survey responses	Correct stress score generated	Correct stress score generated	**Pass**
2	Calculate average stress	Response dataset	Correct average calculated	Correct average calculated	**Pass**

3	Invalid input values	Incorrect data	Error handling message	Error handling message shown	**Pass**

Table No. 7. Stress Calculation Module Test Cases

13.9. Integration Testing Test Cases

Sr. No.	Test Case Description	Modules Involved	Expected Results	Actual Results	Status
1	Login API integration	React + Django API	Successful login authentication	Login successful	**Pass**
2	Survey submission	Frontend + Backend + Database	Data stored successfully	Data stored successfully	**Pass**
3	Stress calculation	Survey + Calculation module	Correct stress results generated	Correct stress results generated	**Pass**
4	Attainment calculation	Master Data + Attainment module	Accurate attainment result	Attainment result was accurate	**Pass**

Table No. 8. Integration Testing Test Cases

13.10. Test Results

The testing process was successfully carried out for all modules of the system. Multiple test cases were executed to verify the correctness of the application. The outputs obtained during testing were compared with the expected outputs, and most test cases passed successfully.

During testing, a few minor issues related to validation and data handling were identified. These issues were resolved during the debugging process, and the system was tested again to confirm the corrections. After completing the testing phase, the system was found to be functioning correctly and efficiently.

CHAPTER 14. CONCLUSION

The OBE Tracking System has been successfully designed and developed to automate and simplify the implementation of Outcome Based Education in academic institutions. The primary objective of this project was to reduce manual workload in attainment calculation and to provide a structured and reliable system for managing academic data.

The system includes modules such as academic configuration, CO–PO–PSO mapping, assessment management, stress survey collection, attainment calculation, and report generation. These modules work together to provide accurate measurement of learning outcomes and overall academic performance.

The frontend of the system is developed using React, which offers a responsive and user-friendly interface. The backend is implemented using Django REST Framework, which efficiently manages business logic and API communication. During development, SQLite was used as the database, and for deployment, PostgreSQL was configured to ensure better scalability and performance. Secure authentication is provided using JSON Web Token (JWT) along with a custom user model for role- based access control.

The developed system meets the defined objectives and provides a secure, accurate, and efficient solution for implementing Outcome Based Education in diploma institutions. The project demonstrates practical use of modern web technologies to solve real academic management challenges.

CHAPTER 15. FUTURE SCOPE

Although the OBE Tracking System is fully functional and successfully meets the current requirements, there is scope for further improvement and enhancement in the future:

- ***Integration with Institutional ERP Systems:*** The system can be integrated with existing institutional ERP platforms so that academic data, such as student records, subject allocation, and examination details, can be automatically synchronized. This will reduce manual data entry and improve efficiency.
- ***Advanced Graphical Dashboards:*** Enhanced dashboards can be added to provide better visualization of CO, PO, and PSO attainment levels. Interactive charts and analytics will help faculty and management understand performance trends more clearly.
- ***Student Self-Monitoring:*** Students can be given access to view and monitor their own performance, enabling them to track their academic progress and identify areas for improvement.
- ***Mobile Application:*** A mobile version of the system can be developed for easier access by both faculty and students, improving usability and convenience.
- ***Artificial Intelligence and Data Analytics:*** AI and advanced data analytics techniques can be incorporated to predict weak learning outcomes, suggest personalized improvement strategies, and provide actionable insights for both students and faculty.
- ***Enhanced Automatic Report Generation:*** Reports can be generated automatically in multiple formats such as PDF and Excel, facilitating easier documentation and submission.
- ***Scalability for Large-Scale Deployment:*** With these improvements, the OBE Tracking System can evolve into a comprehensive academic performance management solution suitable for large-scale educational environments.

CHAPTER 16. REFERENCE

[1] Ministry of Education, Government of India, "National Education Policy 2020," Government of India, New Delhi, India, 2020.

[2] Audrey Mariamo, Caroline Elizabeth Temcheff, Pierre-Majorique Léger, Sylvain Senecal, Marianne Alexandra Lau, "Emotional Reactions and Likelihood of Response to Questions Designed for a Mental Health Chatbot Among Adolescents: Experimental Study", JMIR Hum Factors 2021, vol. 8, iss. 1, e24343, Jan-Mar.

[3] R. Gorter, R. Freeman, S. Hammen, H. Murtomaa, A. Blinkhorn and G. Humphris, "Psychological stress and health in undergraduate dental students: fifth year outcomes compared with first year baseline results from five European dental schools", Eur J Dent Educ, 12, 2008.

[4] Peggy Marie Savage, "Emoji Tracker: Utilizing Data Visualization to Track Student Behavior in Real Time," Richmond Elementary School.

CHAPTER 17. APPENDIX

17.1. APPENDIX A: SOURCE CODE

17.1.1. Introduction:

This appendix contains the source code of key files of the Outcome Based Education (OBE) Tracking System. Complete project source code is available at:

GitHub Repository:

https://github.com/Farheen-H-S/OBE-TRACKING-SYSTEM

17.1.2. Backend Configuration:

17.1.2.1. core/settings.py -

```
from pathlib import Path
from datetime import timedelta
import os
import dj_database_url

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.getenv('DJANGO_SECRET_KEY', 'django-insecure-h*m-
4kgy)@n)j_vcy)9ck6&w-)=!f!mj5gwga(=8ga6y3ncb$g')

DEBUG = os.getenv('DJANGO_DEBUG', 'True') == 'True'

ALLOWED_HOSTS = os.getenv('DJANGO_ALLOWED_HOSTS', 'localhost
127.0.0.1').split()

AUTH_USER_MODEL = 'users.User'

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
```

```
    'corsheaders',
    'users',
    'academics',
    'cis_master',
    'assessments',
    'indirect_attainment',
    'attainment',
    'reports',
    'stress',
    'surveys',
    'teaching_plan8',
    'audit',
    'bulk_upload',
    'notifications',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'core.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
```

```
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'core.wsgi.application'

DATABASES = {
    'default': dj_database_url.config(
        default=f'sqlite:///{BASE_DIR / "db.sqlite3"}',
        conn_max_age=600
    )
}

AUTH_PASSWORD_VALIDATORS = [
    {'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
```

```
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}

SIMPLE_JWT = {
    'USER_ID_FIELD': 'user_id',
    'USER_ID_CLAIM': 'user_id',
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
    'AUTH_HEADER_TYPES': ('Bearer',),
}

CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000",
]

CSRF_TRUSTED_ORIGINS = [f"https://{host}" for host in ALLOWED_HOSTS if host]
if "localhost" in ALLOWED_HOSTS or "127.0.0.1" in ALLOWED_HOSTS:
    CSRF_TRUSTED_ORIGINS.append("http://localhost:3000")

EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
```

```
EMAIL_HOST_USER = 'your-email@gmail.com'
EMAIL_HOST_PASSWORD = 'your-app-password'
DEFAULT_FROM_EMAIL = f'OBE Tracking System <{EMAIL_HOST_USER}>'
```

17.1.2.2. core/urls.py -

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/users/', include('users.urls')),
    path('api/academics/', include('academics.urls')),
    path('api/cis_master/', include('cis_master.urls')),
    path('api/assessments/', include('assessments.urls')),
    path('api/attainment/', include('attainment.urls')),
    path('api/reports/', include('reports.urls')),
    path('api/stress/', include('stress.urls')),
    path('api/surveys/', include('surveys.urls')),
    path('api/teaching-plan/', include('teaching_plan.urls')),
    path('api/audit/', include('audit.urls')),
    path('api/bulk_upload/', include('bulk_upload.urls')),
    path('api/notifications/', include('notifications.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

17.1.3. Frontend:

17.1.3.1. src/App.js (Key Sections) -

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { FilterProvider } from './context/FilterContext';
```

```
const Login = lazy(() => import("./pages/common/login/Login"));
const DashboardRedirect = lazy(() => import("./components/DashboardRedirect"));

function App() {
  return (
    <BrowserRouter>
      <FilterProvider>
        <Suspense fallback={<div>Loading...</div>}>
          <Routes>
            <Route path="/" element={<Login />} />
            <Route path="/dashboard" element={<DashboardRedirect />} />
          </Routes>
        </Suspense>
      </FilterProvider>
    </BrowserRouter>
  );
}

export default App;
```

17.1.4. Services:

17.1.4.1. src/services/authService.js -

```
import api from "../utils/axios";

export const login = async (email, password) => {
    const response = await api.post("/users/login/", { email, password });

    localStorage.setItem("access", response.data.access);
    localStorage.setItem("refresh", response.data.refresh);
    localStorage.setItem("user", JSON.stringify(response.data.user));

    return response.data;
};

export const logout = async (refreshToken) => {
    try {
```

```
        const response = await api.post("users/logout/", { refresh: refreshToken });

        localStorage.removeItem("access");
        localStorage.removeItem("refresh");
        localStorage.removeItem("user");
        sessionStorage.removeItem("access");
        sessionStorage.removeItem("refresh");
        sessionStorage.removeItem("user");

        return response.data;
    } catch (error) {
        console.error("Logout failed:", error);
        throw error;
    }
};

export const getLoggedInUser = () => {
    const user = localStorage.getItem("user");
    return user ? JSON.parse(user) : null;
};
```

17.1.5. Utility Functions:

17.1.5.1. src/utils/auth.js -

```
import api from "../utils/axios";

export const login = async (email, password) => {
    const response = await api.post("/users/login/", { email, password });

    localStorage.setItem("access", response.data.access);
    localStorage.setItem("refresh", response.data.refresh);
    localStorage.setItem("user", JSON.stringify(response.data.user));

    return response.data;
};

export const logout = async (refreshToken) => {
```

```
    try {
        const response = await api.post("/users/logout/", { refresh: refreshToken });

        localStorage.removeItem("access");
        localStorage.removeItem("refresh");
        localStorage.removeItem("user");
        sessionStorage.removeItem("access");
        sessionStorage.removeItem("refresh");
        sessionStorage.removeItem("user");

        return response.data;
    } catch (error) {
        console.error("Logout failed:", error);
        throw error;
    }
};

export const getLoggedInUser = () => {
    const user = localStorage.getItem("user") || sessionStorage.getItem("user");
    if (!user) return null;

    try {
        return JSON.parse(user);
    } catch (err) {
        return null;
    }
};

export const updateLoggedInUser = (userData) => {
    if (!userData) return;

    if (localStorage.getItem("user")) {
        localStorage.setItem("user", JSON.stringify(userData));
    } else if (sessionStorage.getItem("user")) {
        sessionStorage.setItem("user", JSON.stringify(userData));
```

```
    }
};
```

17.1.5.2. src/utils/axios.js -

```
import axios from "axios";

const api = axios.create({
    baseURL: "http://127.0.0.1:8000/api/",
    headers: {
        "Content-Type": "application/json",
    },
});

api.interceptors.request.use(
    (config) => {
        const isPublic =
            config.url.includes("/users/login") ||
            config.url.includes("/users/logout");

        if (!isPublic) {
            const token =
                localStorage.getItem("access") ||
                sessionStorage.getItem("access");
            if (token) {
                config.headers.Authorization = `Bearer ${token}`;
            }
        }
        return config;
    },
    (error) => Promise.reject(error)
);

api.interceptors.response.use(
    (response) => response,
```

```
    (error) => {
        if (error.response && error.response.status === 401) {
            localStorage.clear();
            sessionStorage.clear();
            window.location.href = "/";
        }
        return Promise.reject(error);
    }
);

export default api;
```

17.1.6. Backend Dependencies:

17.1.6.1. requirements.txt -

```
asgiref==3.11.0
Django==6.0
django-cors-headers==4.7.0
djangorestframework==3.16.1
djangorestframework_simplejwt==5.5.1
dj-database-url==2.3.0
gunicorn==23.0.0
numpy==2.2.3
openpyxl==3.1.5
pandas==2.2.3
pillow==12.0.0
psycopg2-binary==2.9.10
PyJWT==2.10.1
python-dotenv==1.2.1
sqlparse==0.5.4
tzdata==2025.2
whitenoise==6.8.2
```

17.1.7. Frontend Dependencies:

17.1.7.1. package.json -

```
{
  "name": "webapp",
  "version": "0.1.0",
  "private": true,
  "proxy": "http://127.0.0.1:8000",
  "dependencies": {
    "@testing-library/dom": "^10.4.1",
    "@testing-library/jest-dom": "^6.9.1",
    "@testing-library/react": "^16.3.0",
    "@testing-library/user-event": "^13.5.0",
    "axios": "^1.13.2",
    "bootstrap": "^5.3.8",
    "jwt-decode": "^4.0.0",
    "react": "^19.2.1",
    "react-bootstrap": "^2.10.10",
    "react-dom": "^19.2.1",
    "react-google-charts": "^5.2.1",
    "react-icons": "^5.5.0",
    "react-router-dom": "^7.11.0",
    "react-scripts": "5.0.1",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  }
}
```

17.2. APPENDIX B: System Screenshots

This appendix presents key user interface screens of the OBE Tracking System. These screenshots demonstrate the overall system workflow, major modules, and core functionalities including outcome mapping, attainment calculation, reporting, and stress analysis.

17.2.1. Login Page:

This screen provides secure user authentication using JWT and redirects users to role - specific dashboards after successful login.

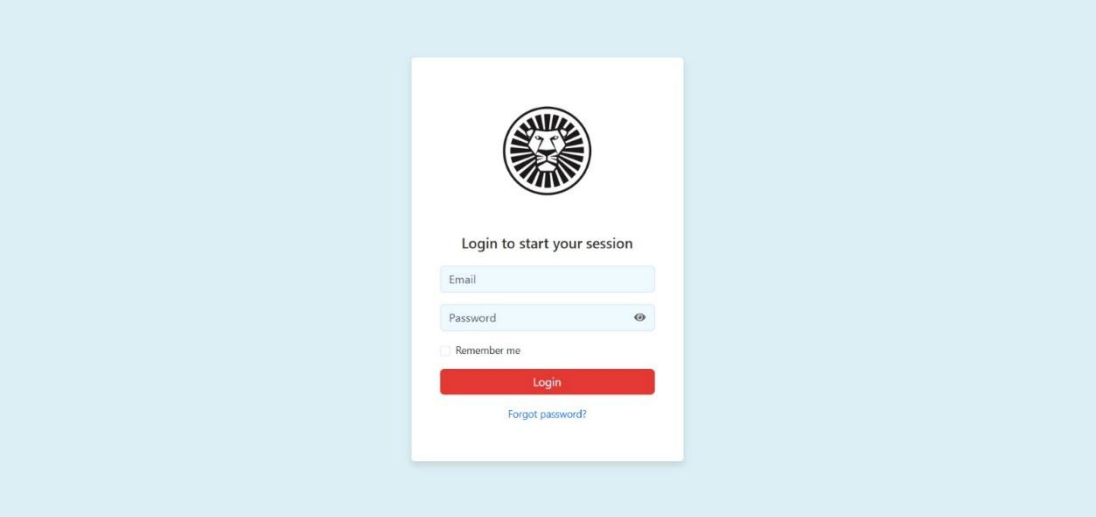


Fig. No. 10. Login Page

17.2.2. Admin Dashboard:

This dashboard provides an overview of system data, including total users, department-wise distribution, and role-based user statistics.

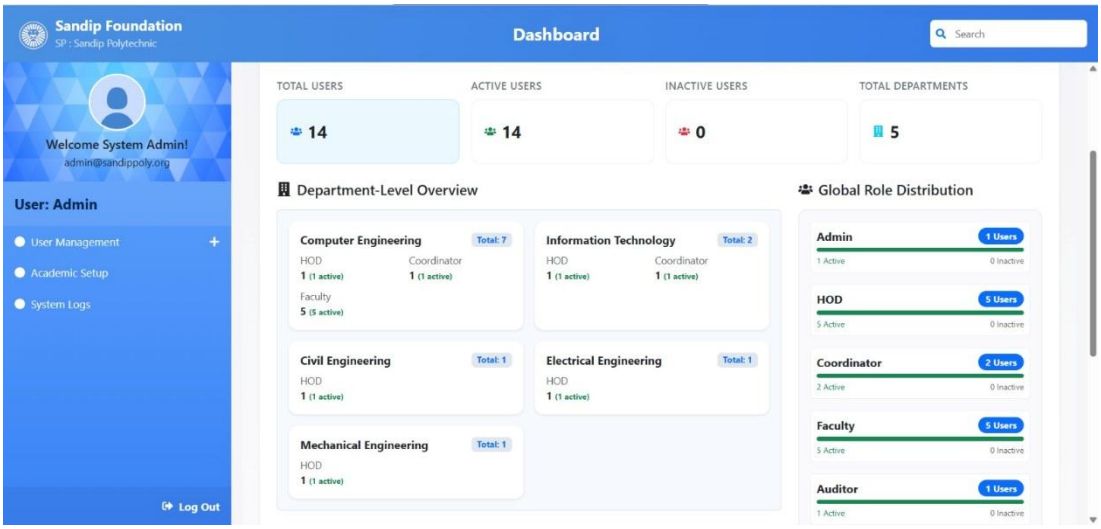


Fig. No. 11. Admin Dashboard

17.2.3. CO–PO–PSO Mapping Interface:

This interface allows mapping of course outcomes with program outcomes and program specific outcomes along with assigned weightages.

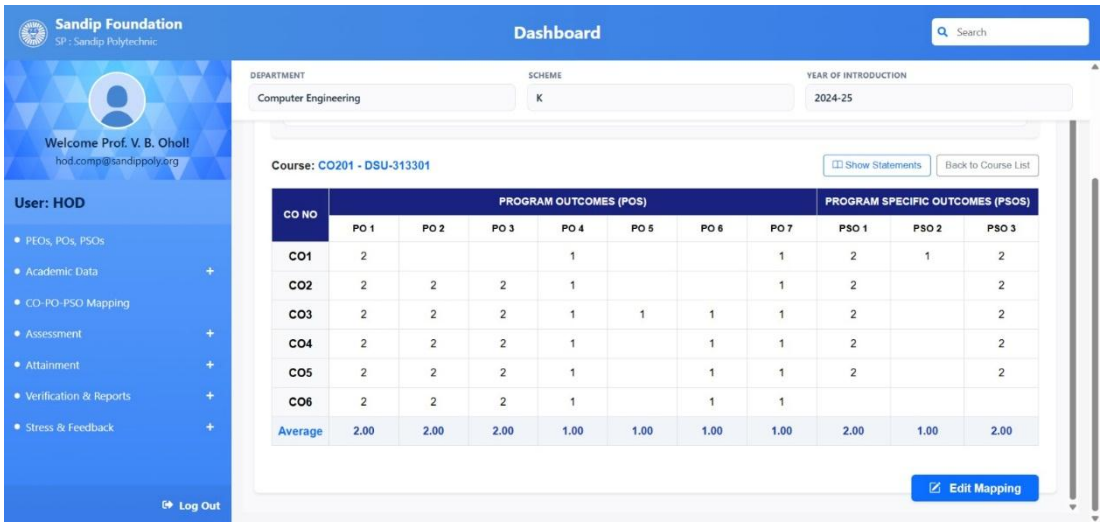


Fig. No. 12. CO–PO–PSO Mapping Interface

17.2.4. CIS Marks Entry:

This screen enables faculty to enter Course Index Sheet (CIS) marks for different tools and ensures validation of input data.

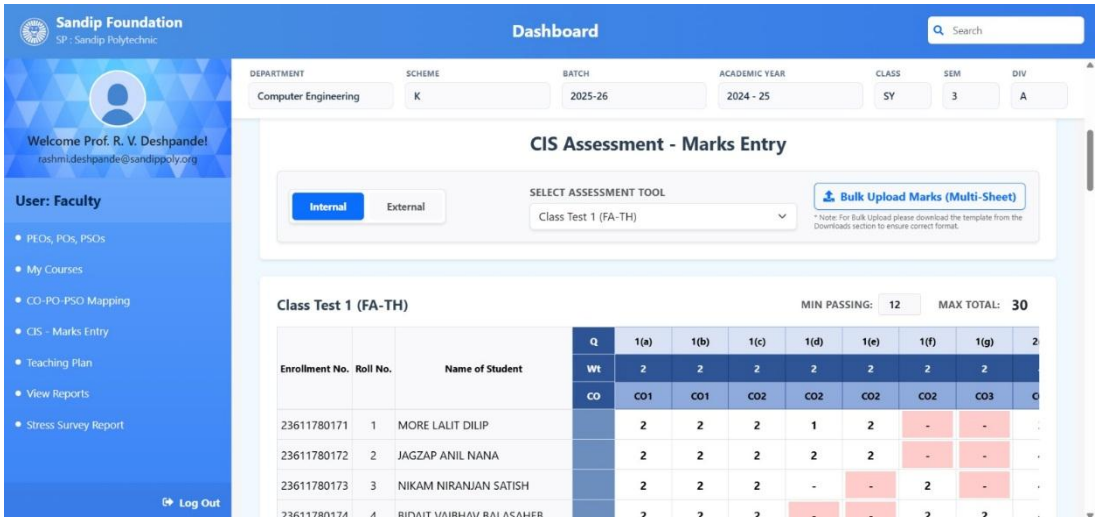
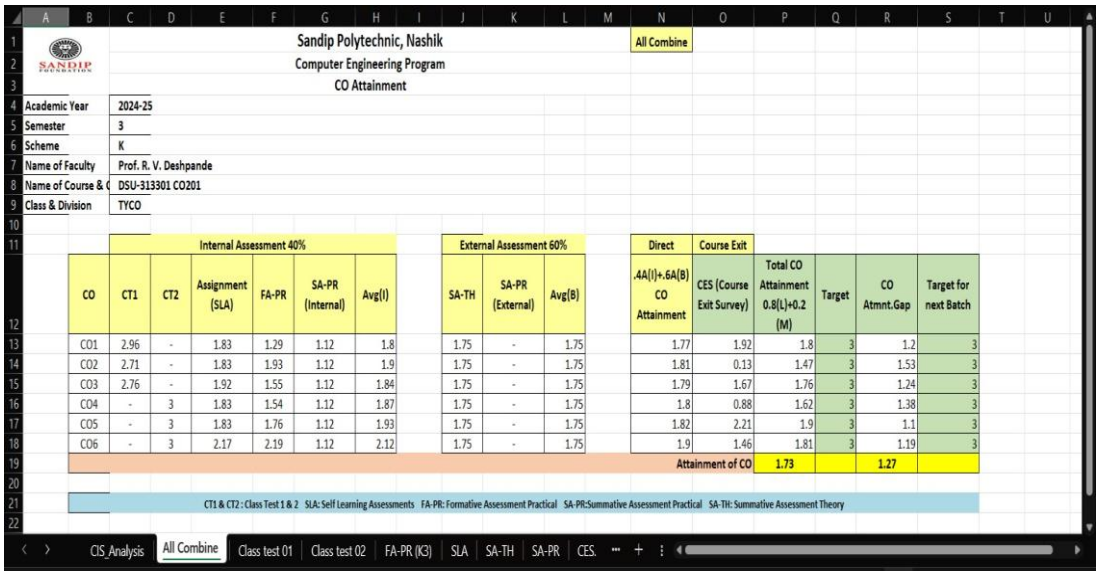


Fig. No. 13. CIS Marks Entry

17.2.5. Direct Attainment Report (Summary):

This report shows calculated CO attainment values based on student performance in internal and external assessments.



Sandip Polytechnic, Nashik
Computer Engineering Program
CO Attainment

All Combine

Academic Year	2024-25
Semester	3
Scheme	K
Name of Faculty	Prof. R. V. Deshpande
Name of Course & C	DSU-313301 CO201
Class & Division	TYCO

	Internal Assessment 40%						External Assessment 60%			Direct	Course Exit				
CO	CT1	CT2	Assignment (SLA)	FA-PR	SA-PR (Internal)	Avg(I)	SA-TH	SA-PR (External)	Avg(B)	.4A(I)+.6A(B) CO Attainment	CES (Course Exit Survey)	Total CO Attainment 0.8(L)+0.2 (M)	Target	CO Atmnt.Gap	Target for next Batch
CO1	2.96	-	1.83	1.29	1.12	1.8	1.75	-	1.75	1.77	1.92	1.8	3	1.2	3
CO2	2.71	-	1.83	1.93	1.12	1.9	1.75	-	1.75	1.81	0.13	1.47	3	1.53	3
CO3	2.76	-	1.92	1.55	1.12	1.84	1.75	-	1.75	1.79	1.67	1.76	3	1.24	3
CO4	-	3	1.83	1.54	1.12	1.87	1.75	-	1.75	1.8	0.88	1.62	3	1.38	3
CO5	-	3	1.83	1.76	1.12	1.93	1.75	-	1.75	1.82	2.21	1.9	3	1.1	3
CO6	-	3	2.17	2.19	1.12	2.12	1.75	-	1.75	1.9	1.46	1.81	3	1.19	3
											Attainment of CO	1.73		1.27	

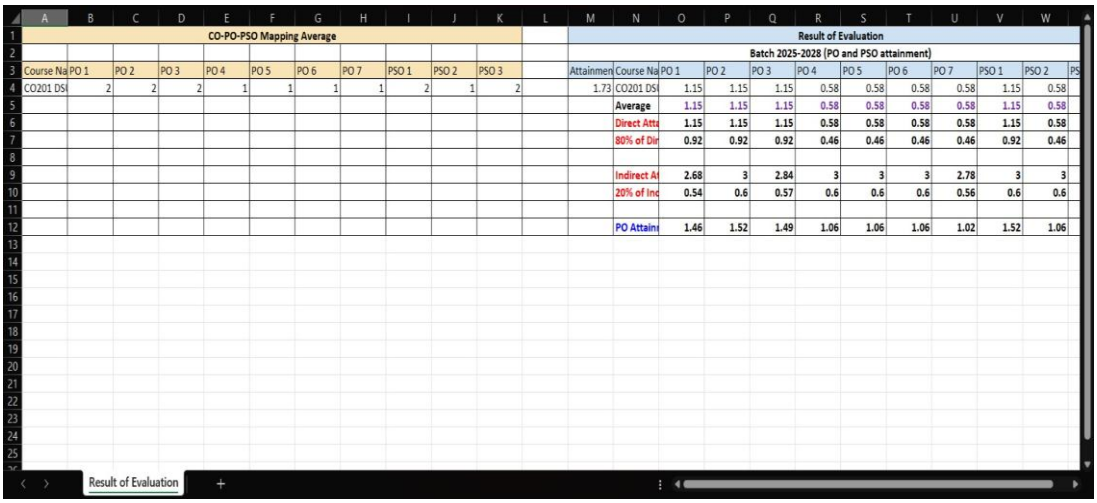
CT1 & CT2 : Class Test 1 & 2 SLA: Self Learning Assessments FA-PR: Formative Assessment Practical SA-PR:Summative Assessment Practical SA-TH: Summative Assessment Theory

CIS_Analysis | All Combine | Class test 01 | Class test 02 | FA-PR (K3) | SLA | SA-TH | SA-PR | CES.

Fig. No. 14. Direct Attainment Report (Summary)

17.2.6. PO/PSO Attainment Report (Result of Evaluation):

This report presents aggregated attainment of program outcomes and program specific outcomes derived from CO attainment.



CO-PO-PSO Mapping Average

Course Na	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PSO 1	PSO 2	PSO 3
CO201 DS	2	2	2	1	1	1	1	2	1	2

Result of Evaluation

Batch 2025-2028 (PO and PSO attainment)

Attainmen	Course Na	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PSO 1	PSO 2	PS
1.73	CO201 DS	1.15	1.15	1.15	0.58	0.58	0.58	0.58	1.15	0.58	
	Average	1.15	1.15	1.15	0.58	0.58	0.58	0.58	1.15	0.58	
	Direct Atta	1.15	1.15	1.15	0.58	0.58	0.58	0.58	1.15	0.58	
	80% of Dir	0.92	0.92	0.92	0.46	0.46	0.46	0.46	0.92	0.46	
	Indirect At	2.68	3	2.84	3	3	3	2.78	3	3	
	20% of Inc	0.54	0.6	0.57	0.6	0.6	0.6	0.56	0.6	0.6	
	PO Attainr	1.46	1.52	1.49	1.06	1.06	1.06	1.02	1.52	1.06	

Result of Evaluation

Fig. No. 15. PO/PSO Attainment Report (Result of Evaluation)

17.2.7. Backtracking Module:

This module enables reverse mapping from PO to CO and associated assessments for detailed performance analysis.

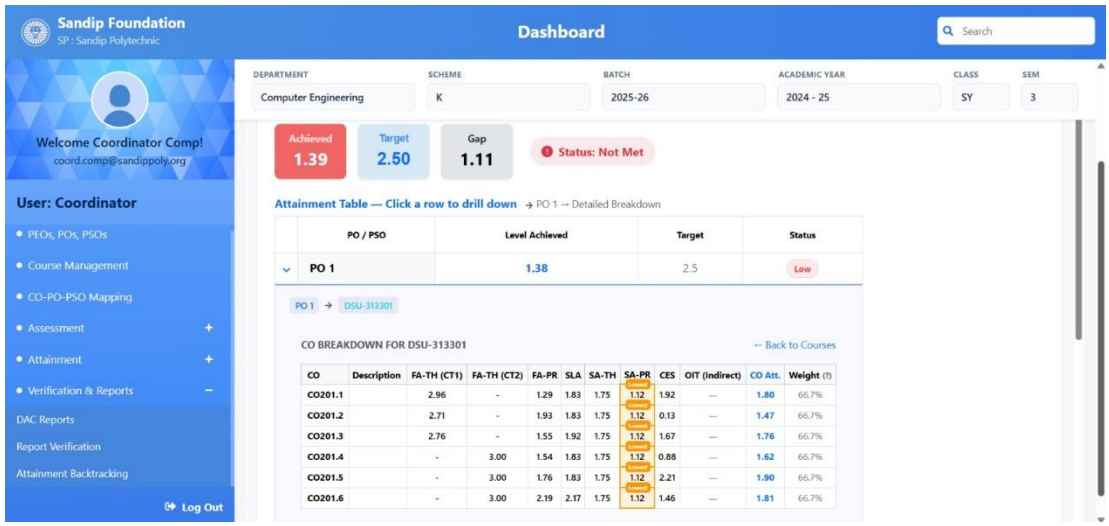


Fig. No. 16. Backtracking Module

17.2.8. Stress Survey Result (Overall Preview):

This screen displays analyzed student stress levels, including overall average stress, domain-wise analysis, and identification of major stress factors based on survey responses.

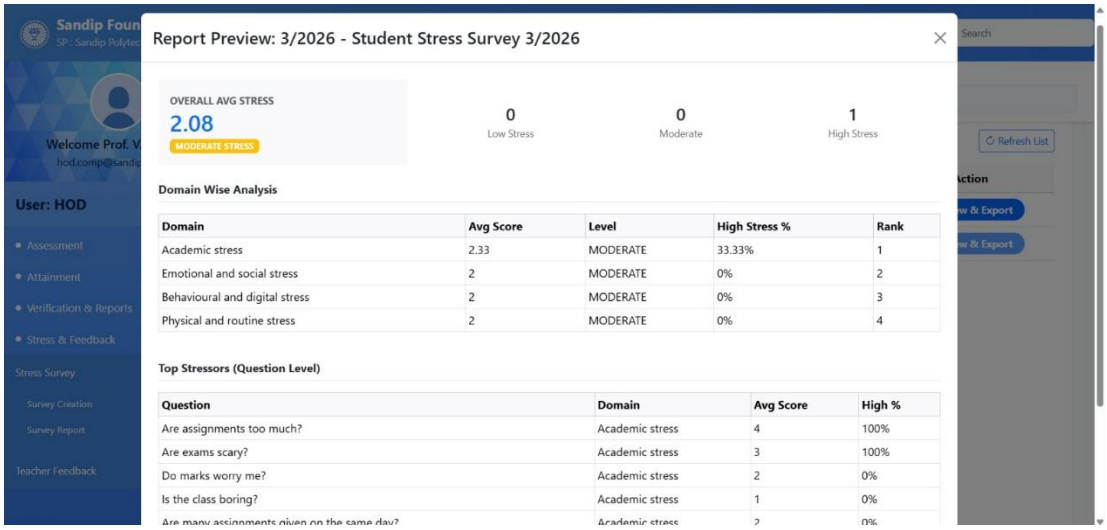


Fig. No. 17. Stress Survey Result (Overall Preview)